

UCC 152-2

Introduction to security in DevOps

Workshop 3: Dynamic Security Analysis of
DigiBank at Runtime

Cloud Computing Professional License

University of the Mountains

By **Engineer Willy Damtchou**

July 2026

Part 1: General framework of workshop 3

1.1 General context of the workshop

As part of the UCC152-2 Introduction to Security in DevOps course, Workshop 3 is the logical and direct continuation of Workshop 2, which focused on the static security analysis of the DigiBank project. After examining the source code, identifying the main weaknesses visible upon reviewing the project, and applying an initial set of fixes, it now becomes necessary to examine the application not just as a structured software artifact, but as a real, functioning system exposed to requests, errors, unexpected inputs, and potentially malicious use.

This transition is essential from both a pedagogical and methodological standpoint. An application may indeed appear cleaner after a SAST workshop, successfully compile, pass some of its unit tests, and present a reasonably sound code structure, while still exhibiting problematic behaviors at runtime: overly detailed error messages, insufficient access controls, inconsistent HTTP responses, validations that can still be bypassed via the API, or incomplete session management. Workshop 3 is specifically designed to bring these externally visible weaknesses to light.

1.2 Pedagogical positioning of the workshop

Workshop 3 shifts the student's perspective from that of an internal analyst to that of an external observer. In Workshop 2, the student examined the code, configuration files, dependencies, and certain structural choices to identify weaknesses before execution. In Workshop 3, they must adopt a different approach: they query the application through its actual entry points, analyze the responses returned, observe the behavior of the endpoints, and verify whether the system correctly defends itself against legitimate, invalid, or hostile requests.

This shift in perspective is central to the complementarity between SAST and DAST presented in the course. SAST allows us to identify risk in the code; DAST allows us to observe risk in the application's observable behavior. Students thus learn that robust security cannot rely on a single source of observation. It must be built from complementary perspectives: one focused on the internal structure, the other on the system's actual exposure.



1.3 Purpose of workshop 3

The goal of Workshop 3 is to conduct a dynamic security analysis campaign of DigiBank at runtime, and then to correct the vulnerabilities revealed by this external observation. In other words, it's neither about repeating Workshop 2, nor about passively executing a few HTTP calls without any reasoning. It's about designing a true DAST (Dynamic Security Analysis) approach, focused on the detection, qualification, remediation, and revalidation of weaknesses visible at runtime.

The application expected at the end of this workshop should therefore be a version of DigiBank that is not only cleaner in terms of code, but also more robust in its runtime behavior. API responses must be better controlled, unauthorized access scenarios must be properly handled, error messages must avoid unnecessary disclosures, and the management of exchanges with API clients must be more consistent from a security perspective.

1.4 Purpose of workshop 3

This workshop illustrates another fundamental dimension of DevSecOps: security is not built solely through prevention upstream, but also through verification in the execution context. From a purely theoretical standpoint, one might consider that once the code is corrected, security is achieved. However, this course demonstrates that a system must be tested against its real-world behavior, as some weaknesses only become apparent when the application receives requests, generates errors, exchanges session data, or exposes its APIs to an external client.

Workshop 3 thus concretely illustrates the idea that application security must be continuously verified. This logic extends the Shift Left approach without contradicting it. The Shift Left encourages early detection; DAST emphasizes that early prevention must be complemented by runtime validation to prevent seemingly fixed code from still exhibiting risky behaviors once deployed or even simply started in a test environment.

1.5 Why DigiBank is a good support for a DAST workshop

The DigiBank project is a particularly relevant case study for a DAST workshop because it combines several characteristics typical of modern systems exposed to real application risks. It is a digital banking application handling sensitive data, exposing REST endpoints, processing critical operations such as account inquiries, beneficiary management, and transfer execution, all within a modern application service architecture.

Such an application allows for concrete testing of key aspects of dynamic security: the quality of HTTP responses, the robustness of validations, endpoint protection, the consistency of access denials, the behavior of error messages, the control of technical information returned to the client, and, more broadly, the observable robustness of the system in the face of normal or malicious interactions. DigiBank also serves as the central theme of the course, which strongly reinforces the pedagogical continuity between workshops.

1.6 Why DigiBank is a good support for a DAST workshop

In this workshop, dynamic analysis refers to observing DigiBank's security during its actual execution, based on its exposed interfaces and observable behaviors. The focus is no longer on examining what the code appears to do, but on verifying what the application actually does when a client interacts with it.

This analysis involves HTTP calls, targeted test scenarios, valid and invalid requests, observation of status codes, response bodies, error messages, access denials, session behavior, and the effects of certain parameter manipulations. DAST thus seeks to identify external symptoms of security weaknesses, even when these are not immediately apparent from simply reading the code.

1.7 General objective of the workshop

The overall objective of Workshop 3 is to detect, analyze, prioritize, correct, and revalidate security vulnerabilities observed at runtime in the DigiBank application, using a DAST approach consistent with the concepts studied in the course. This objective requires that students be able not only to execute tests, but also to design them intelligently, interpret their results, correct the root causes of the identified anomalies, and demonstrate that the remediation has effectively improved the system's external behavior.

1.8 Specific Objectives

At the end of this workshop, the student should be able to:

- **A3.1:** Explain the role of DAST in a DevSecOps approach and link it to the validation during system execution.
- **A3.2:** Design a security-oriented API testing campaign on DigiBank.
- **A3.3:** Analyze HTTP responses, error messages, and statuses returned by the application.
- **A3.4:** Verify the authentication, authorization and session mechanisms when exposed in the test environment.
- **A3.5:** Detect observable vulnerabilities on endpoints, inputs, errors, and inconsistent behavior.
- **A3.6:** Correct the problems revealed by the dynamic tests without breaking DigiBank's business logic.
- **A3.7:** Replay the scenarios after correction in order to demonstrate the effectiveness of the remediations.
- **A3.8:** Produce technical deliverables outlining the test campaign, the vulnerabilities observed and the evidence of correction.

1.9 Scope of the analysis in workshop 3

The scope of this workshop covers vulnerabilities or weaknesses that can reasonably be observed from the outside while DigiBank is running. This includes, in particular, REST endpoints, parameters passed to APIs, responses generated by controllers, behaviors related to errors, validations and unauthorized access, as well as visible symptoms of inadequate session management.

However, the primary objective of this workshop is not to exhaustively analyze dependencies, harden container images, or deeply secure packaging artifacts. These topics are primarily addressed in the subsequent section dedicated to container and dependency security. This distinction is important because it clearly differentiates between the dynamic observation of the system in operation and the analysis of its accompanying artifacts.

1.10 Types of vulnerabilities targeted

Within the DigiBank context, the vulnerabilities or symptoms sought during the dynamic analysis may include:

- Overly detailed error messages returned by the API.
- Responses revealing unnecessary technical information about the environment or implementation.
- Insufficient validations visible from the endpoints.
- Insufficient or circumventable access controls on some roads.
- Imprecise, inconsistent, or incomplete session management.
- Abnormal behavior in response to unexpected, missing, falsified or malicious parameters.
- Inconsistencies between the expected business logic and the behavior observed through the API.

1.11 Types of vulnerabilities targeted

The tools selected for Workshop 3 must allow for the execution, automation, observation, and documentation of dynamic security tests on DigiBank. They should not be presented as an end in themselves, but as instruments to support a reasoned experimental approach. The recommended tools are as follows:

- **IntelliJ IDEA** or an equivalent IDE, to open the project, run it, correct it and quickly relaunch local validations.
- **JDK Java** and **Maven** , to build and launch DigiBank in a test environment.
- **Spring Boot** , which constitutes the application execution framework of the project.
- **Postman** , to design and execute manual and semi-automated test scenarios on endpoints.
- **Newman** , to automatically run a Postman collection and produce test campaign reports.
- **JUnit** or **Spring Boot integration tests**, if certain checks need to be completed on the project side to confirm a remediation.
- **OWASP ZAP** optionally, if the student wishes to enrich the workshop with an additional dynamic scanning dimension.
- **Git** and **GitHub** , to track corrections and maintain traceability of changes.

1.12 Expected result at the end of part 1

At the end of this first part, the student should understand what dynamic security analysis (DAST) actually is, why it follows the SAST workshop, and how it complements the static observation of the DigiBank project. They should also be able to explain that the goal of DAST is not simply to "run queries," but to observe the system's external behavior in a structured way in order to identify vulnerabilities that only become apparent in an execution context.

The next part can then move into the concrete preparation of the dynamic test environment: starting the application, choosing scenarios, preparing demonstration accounts and organizing the observation campaign.

Part 2: Practical Preparation of DigiBank's DAST Environment

In this second part of Workshop 3, the objective is not yet to immediately delve into the detailed identification of dynamic vulnerabilities, but rather to prepare a sufficiently clean, stable, and reproducible execution and testing environment so that security observations are meaningful. This step is fundamental, because a DAST workshop is not about randomly running a scanner on an application started under uncertain conditions. On the contrary, it is essential to first understand the state of the system under test, know the exposed endpoints, have a usable dataset, and properly instrument the tools that will be used to observe the actual behavior of DigiBank in operation.

This approach is consistent with the course progression. Workshop 1 focused on building a modular, executable, and documented application framework. Workshop 2 then taught participants how to examine this framework from the perspective of its code, dependencies, and static configurations. Workshop 3 naturally extends this work by focusing on the execution level, that is, the observable behavior of the application as it receives HTTP requests, exposes its Swagger documentation, interacts with its database, and responds to normal and abnormal input.

2.1 Role of this phase in the workshop

Before detecting vulnerabilities through dynamic analysis, the student must ensure that DigiBank boots correctly, that the PostgreSQL database is available, that the primary endpoints are responding, and that the testing environment is sufficiently controlled to avoid confusing installation errors with genuine security weaknesses. This requirement is crucial because a poorly prepared DAST often produces misleading results: false negatives if the application doesn't actually function, false positives if certain routes fail for purely technical reasons, or non-reproducible results if the database state changes uncontrollably between runs.

This phase therefore has a threefold purpose. First, it restores DigiBank to a clean, functional state after the corrections made in Workshop 2. Second, it prepares the dynamic testing tools, including the manual HTTP client, the automated scanning tool, and any necessary test scripts. Finally, it establishes the conditions for reproducing the practical exercise, so that two students or two successive runs can observe roughly the same attack surface and interpret the results on a comparable basis.

2.2 Role of this phase in the workshop

The student must start from a workstation already operational for the Java/Spring Boot environment used in DigiBank, in accordance with the standardized technical choices made in Workshop 1: Java 17, Maven, IntelliJ IDEA, PostgreSQL, Docker, Git, and a multi-module Spring Boot application managed by a Maven parent. For Workshop 3, students must also have tools adapted for dynamic testing, as the DAST cannot be limited to local execution of the backend; it must also allow for the observation and manipulation of HTTP requests and responses.

The components to install or check are as follows:

- **Java JDK 17.**
- **Maven 3.9.x** or compatible version.
- **IntelliJ IDEA Community or Ultimate .**
- **Git .**
- **PostgreSQL** or PostgreSQL via Docker.
- **Docker Desktop** or Docker Engine with Docker Compose.

- A modern web browser .
- **Postman** for manual testing and building a collection of calls.
- **OWASP ZAP** for automated or semi-automated dynamic testing.
- **Newman** to automatically replay a Postman collection from the command line.
- **GitHub access** is required if you wish to prepare for the subsequent automation of tests in a pipeline.

The minimum verification commands to execute in a terminal are as follows:

```
Java -version
mvn -version
git --version
docker --version
docker compound version
psql --version
```

2.3 Tools selected for the DAST environment

In this section, it is important not to present the tools as independent elements, but as building blocks of a single dynamic observation chain. Each tool has a distinct function in the testing strategy applied to DigiBank.

The tools selected are as follows:

- **IntelliJ IDEA:** allows you to open, modify and run DigiBank, as well as view startup logs and the classes involved in the observed behaviors.
- **Maven:** allows you to rebuild the application, run existing tests and ensure that the version submitted to DAST is consistent.
- **Spring Boot:** provides the runtime of the application to be tested.
- **PostgreSQL:** hosts the data manipulated by DigiBank during dynamic tests.
- **Swagger UI / OpenAPI:** provides an initial mapping of exposed endpoints and enables rapid controlled testing.
- **Postman:** is used to send manual or semi-structured requests, to test normal and abnormal cases, and to build a reusable collection.

- **OWASP ZAP:** is used to perform a crawl, traffic observation and passive or active scans on the running application.
- **Newman:** allows you to automatically replay a Postman collection in a repeatable verification logic.
- **Docker / Docker Compose:** facilitate the setup of a reproducible environment for DigiBank and PostgreSQL.

Within the course's pedagogical framework, Postman and Swagger initially serve a comprehension and preparatory role, while OWASP ZAP introduces the observation and scanning logic specific to DAST. This connection is important: scanning an application is not useful if its functional aspects are not yet understood.

2.4 Clean reopening of the DigiBank project

The project used for workshop 3 must be the version from workshop 2, meaning a version that has already been reviewed, statically analyzed, and corrected on the main points identified. The course emphasizes that DAST (Dynamic Analysis of Static Analysis) comes after SAST (Static Analysis of Static Analysis) in the pedagogical progression; it does not replace static analysis, but extends it by observing the actual behavior of the already improved version.

In IntelliJ IDEA, the recommended procedure is as follows:

1. Open IntelliJ IDEA.
2. Choose Open.
3. Select the root folder digibank-parent.
4. Verify that the project is correctly recognized as a multi-module Maven project.
5. Wait for the project to be fully indexed.
6. Open the Maven tab.
7. Click on Reload All Maven Projects.
8. Check for the presence of the common-module, customer-module, account-module, transfer-module and digibank-web modules.
9. Verify that the Spring Boot dependencies are resolved correctly and that no Maven errors appear in the IDE.

At this stage, a control build must be executed from the project root:

```
CD digibank-parent
```

```
mvn clean install
```

This command has a simple but essential role: it confirms that the DigiBank version submitted for dynamic testing still compiles, that the existing tests pass, and that the application is not in a partially broken state even before DAST is launched. Labs 1 and 2 already present this command as a standard global check of the multi-module project; it therefore makes sense to keep it here as the starting point for Lab 3.

2.5 Clean reopening of the DigiBank project

As in workshop 2, it is recommended to create a Git restore point before adding Postman collection files, scan configurations, or dynamic test scripts. This practice is consistent with the DevSecOps logic of traceability and reproducibility that runs throughout the entire process.

```
git status  
  
git add .  
  
git commit -m "Baseline before DAST environment setup"
```

This commit clearly distinguishes the state of DigiBank before DAST preparation and the state of the project after implementation of the dynamic testing tools. It will be useful if the student needs to revert to previous versions or justify the changes introduced specifically for Workshop 3.

2.6 Verification of application configuration before dynamic testing

DAST assumes that the application actually runs in a consistent environment. Therefore, before any scan, the Spring Boot configuration used to run DigiBank locally must be verified. Workshop 1 already structured the application around a digibank-web module, an application.yml file, Swagger/OpenAPI, and a standard Spring Boot startup; Workshop 3 reuses this foundation as an observation point.

The digibank-web/src/main/resources/application.yml file must contain at least a configuration of this type:

```
spring :  
  
  application :  
  
    name : digibank  
  
  profiles :  
  
    active : ${SPRING_PROFILES_ACTIVE:dev}  
  
  jpa :  
  
    hibernate :  
  
      ddl-auto : validate  
  
      show-sql : false  
  
      open-in-view : false  
  
      properties :  
  
        hibernate :  
  
          SQL format : true  
  
          jdbc :  
  
            time zone : UTC  
  
  thymeleaf :  
  
    cache : true  
  
SQL :
```

```
init :  
    mode : never  
  
server :  
    port : ${SERVER_PORT:8080}  
    error :  
        include-message : never  
        include-binding-errors : never  
        include-stacktrace : never  
  
springdoc :  
    api-docs :  
        enabled : false  
    swagger-ui :  
        enabled : false  
  
logging :  
    level :  
        root : INFO  
        org.springframework.web : INFO  
        org.hibernate.SQL : INFO  
        org.hibernate.type.descriptor.sql.BasicBinder : INFO  
  
error :
```

```
include-message : never

include-binding-errors : never

include-stacktrace : never
```

File digibank-web/src/main/resources/application-dev.yml

```
spring :

  datasource :

    url : ${SPRING_DATASOURCE_URL}

    username : ${SPRING_DATASOURCE_USERNAME}

    password : ${SPRING_DATASOURCE_PASSWORD}

    driver-class-name : org.postgresql.Driver


  jpa :

    hibernate :

      ddl-auto : validate

    show-sql : false

    open-in-view : false

    properties :

      hibernate :

        SQL format : true

      jdbc :

        time zone : UTC


  flyway :
```

```
enabled : true

baseline-on-migrate : true

locations : classpath:db/migration/postgresql

default-schema : public

schemas : public
```

File digibank-web/src/main/resources/application-test.yml

```
spring :

  datasource :

    url : jdbc:h2:mem:digibanktestdb

    driver-class-name : org.h2.Driver

    username : sa

    password :


  jpa :

    hibernate :

      ddl-auto : create-drop

      show-sql : false


  flyway :

    enabled : true

    baseline-on-migrate : true

    locations : classpath:db/migration/h2
```



This configuration has several advantages in the DAST context. It fixes the application's exposure port, maintains access to Swagger UI for endpoint mapping, limits the detailed exposure of client-side errors, and provides logs useful for local observation during testing.

2.7 Preparing the PostgreSQL database

For a DAST workshop, the database should not be a secondary element. Since DigiBank handles customers, accounts, and transfers, a stable dataset is essential for testing application behavior under realistic conditions. Workshop 1 already demonstrated that PostgreSQL serves as the project's reference relational database, and that DigiBank relies on this persistent database for its business operations.

If PostgreSQL is installed locally, verify that the database exists with the project's educational parameters:

- Base: **digibankdb**
- User: **digibank**
- Password: **digibank123**

If necessary, recreate the environment from psql:

```
CREATE DATABASE digibankdb ;

CREATE USER digibank WITH ENCRYPTED PASSWORD 'digibank123' ;

GRANT ALL PRIVILEGES ON DATABASE digibankdb TO digibank;
```

If the student prefers to use Docker, which is often more reproducible in an educational context, the following command can be used:

```
docker run -d \

--name digibank-postgres \

--restart unless stopped \

-e POSTGRES_DB=digibankdb \
```

```
-e POSTGRES_USER=digibank \  
  
-e POSTGRES_PASSWORD=digibank123 \  
  
-p 5432:5432 \  
  
-v digibank-postgres-data:/var/lib/postgresql/data \  
  
postgres:16
```

Then check:

```
docker ps  
  
docker logs digibank-postgres
```

This step is important for DAST, because an application that fails on a request due to an unavailable database does not illustrate an exploitable security weakness; it simply shows a failing test environment.

2.8 Recommended use of Docker Compose for a reproducible environment

Even though the application can be launched directly from the IDE, for a DAST workshop, it's preferable to have a more stable environment that more closely resembles a real-world deployment. Workshop 1 already introduced Docker and Docker Compose as tools for industrializing the execution of DigiBank; Workshop 3 can build on this foundation to standardize the testing environment.

Create the following docker-compose.yml file in the project root directory:

```
version : '3.8'  
  
services :  
  
  postgres :  
  
    image : postgres:16
```

```

container_name : digibank-postgres

environment :

    POSTGRES_DB : digibankdb

    POSTGRES_USER : digibank

    POSTGRES_PASSWORD : digibank123

ports :
- "5432:5432"

volumes :
- postgres-data:/var/lib/postgresql/data

health check :

    test : [ "CMD-SHELL" , "pg_isready -U digibank" ]

    interval : 10s

    Timeout : 5s

    retries : 5

app :

    build : .

    container_name : digibank-app

    ports :
- "8080:8080"

    environment :

                                SPRING_DATASOURCE_URL :
jdbc:postgresql://postgres:5432/digibankdb

    SPRING_DATASOURCE_USERNAME : digibank

```

```
    SPRING_DATASOURCE_PASSWORD : digibank123

    depends_on :

        postgres :

            condition : service_healthy

volumes :

    postgres-data :
```

This file allows the application and its database to be launched in a more consistent environment, with a single command:

```
docker compound up --build
```

The basic commands to know are:

```
docker compound up --build

docker compound up -d --build

docker compound ps

docker compound logs -f

docker compound down

docker compound down -v
```

This approach is particularly useful for DAST, as it avoids excessive differences between student workstations and makes scans more reproducible.

2.9 Functional verification before DAST instrumentation

Before opening Postman or OWASP ZAP, you must verify that DigiBank responds normally in a simple usage scenario. This step is essential because the dynamic analysis must start from a truly usable application, not a backend that is still partially unstable.

Launch the application either via IntelliJ IDEA or with Maven:

```
mvn spring-boot:run -pl digibank-web
```

Or via Docker Compose:

```
docker compose up --build
```

Next, check in the browser:

- <http://localhost:8080>
- <http://localhost:8080/swagger-ui.html>
- <http://localhost:8080/api-docs>

The expected checks are as follows:

- The DigiBank homepage is displayed.
- Swagger UI is accessible.
- The endpoints of the customer, account and transfer domains are visible.
- Simple calls from Swagger return consistent JSON responses.

This step confirms that the application exposure surface does indeed exist and that it can now be subjected to further testing.

2.10 Preparing a demonstration dataset

A DAST workshop becomes much more relevant if the application already contains data on which to test query, creation, search, and transfer behaviors. Workshop 1 already proposed an initialization mechanism using CommandLineRunner, followed by more reproducible logic using Flyway with versioned SQL scripts.

If Flyway has been implemented in the project, check for the presence of the following scripts in digibank-web/src/main/resources/db/migration/:

- V1_create_schema.sql
- V2_insert_seed_data.sql

A consistent example of an initial data script is as follows:

```
INSERT INTO customers (id, full_name, email, phone_number,
national_id, created_at)

VALUES

( 1 , 'Alice Ndzi' , 'alice@digibank.com' , '237600000001' ,
'CNI000001' , CURRENT_TIMESTAMP),

( 2 , 'Brice Tchoumi' , 'brice@digibank.com' , '237600000002' ,
'CNI000002' , CURRENT_TIMESTAMP),

( 3 , 'Clarisse Mvondo' , 'clarisse@digibank.com' ,
'237600000003' , 'CNI000003' , CURRENT_TIMESTAMP);

INSERT INTO accounts (id, account_number, balance, account_type,
customer_id, created_at)

VALUES

( 1 , 'DB-000001' , 150000.00 , ' CURRENT ' , 1 ,
CURRENT_TIMESTAMP),

( 2 , 'DB-000002' , 90000.00 , ' SAVINGS ' , 2 ,
CURRENT_TIMESTAMP),

( 3 , 'DB-000003' , 125000.00 , ' CURRENT ' , 3 ,
CURRENT_TIMESTAMP);
```

```

INSERT INTO transfers (id, source_account_id,
destination_account_id, amount, description, transfer_date,
status )

VALUES

( 1 , 1 , 2 , 5000.00 , ' Initial test transfer' ,
CURRENT_TIMESTAMP, 'SUCCESS' ),

( 2 , 2 , 3 , 3500.00 , ' Seed transfer' , CURRENT_TIMESTAMP,
'SUCCESS' );

```

The value of this data is twofold. Firstly, it allows for immediately meaningful calls in Swagger, Postman, or ZAP. Secondly, it guarantees better reproducibility of dynamic tests across multiple executions of the practical exercise.

2.11 Postman Installation and Preparation

Postman plays a transitional role between functional API understanding and more systematic security testing. It allows students to build a clear collection of calls, verify nominal behavior, and then gradually introduce variations in parameters, request bodies, or identifiers to observe how DigiBank reacts.

After installing Postman, create a new collection named:

DigiBank DAST Collection

Then define an environment variable:

- **baseUrl = http://localhost:8080**

Next, create the following minimal queries.

Query 1: Customer List

- Method: **GET**
- URL: **{{baseUrl}}/api/customers**

Query 2: List of accounts

- Method: **GET**
- URL: **{{baseUrl}}/api/accounts**

Request 3: Creating a client

- Method: **POST**
- URL: **{{baseUrl}}/api/customers**
- Headers: **Content-Type: application/json**
- Body:

```
{  
  
  "fullName" : "Jean Testeur" ,  
  
  "email" : "jean.testeur@digibank.com" ,  
  
  "phoneNumber" : "237699999999" ,  
  
  "nationalId" : "CNI999999"  
}
```

Request 4: Creating an account

- Method: **POST**
- URL: **{{baseUrl}}/api/accounts**
- Headers: **Content-Type: application/json**
- Body:

```
{  
  
  "customerId" : 1 ,  
  
  "accountType" : "CURRENT" ,  
  
  "initialBalance" : 25000.00  
}
```

Request 5: Transfer

- Method: **POST**
- URL: **{{baseUrl}}/api/transfers**
- Headers: **Content-Type: application/json**
- Body:

```
{  
  
  "sourceAccountId" : 1 ,  
  
  "destinationAccountId" : 2 ,  
  
  Amount : 1500.00  
  
}
```

These queries serve as a baseline. Before any abnormal testing, it's essential to confirm they function correctly in a standard scenario. Only after this verification can variations be tested to highlight insufficient validations, overly verbose error messages, or fragile business behaviors.

2.12 Recommended export of a versioned Postman collection

In a DevSecOps approach, it's beneficial to ensure that manual or semi-automated test scenarios aren't confined to a student's local tool. Therefore, it's recommended to export the Postman collection and version it in the repository so that it can be replayed or reviewed by others.

Create a directory at the root of the project:

```
mkdir -p dast/postman
```

Then export:

- DigiBank-DAST-Collection.postman_collection.json
- DigiBank-Local.postman_environment.json

The expected directory becomes, for example:

```
dast/  
  
postman/  
  
    DigiBank-DAST-Collection.postman_collection.json  
  
    DigiBank-Local.postman_environment.json
```

2.13 OWASP ZAP Installation and Preparation

OWASP ZAP is the central dynamic scanning tool in this workshop. Its role is not to replace human analysis, but to help map the application, observe HTTP exchanges, perform passive scans and, subsequently, launch more active tests on the exposed surface.

After installing ZAP, the recommended procedure is as follows:

1. Open **OWASP ZAP**.
2. Choose a persistent or temporary session.
3. Verify that the interface correctly displays the site tree, query history, alerts, and scan tabs.
4. Ensure that the target of the workshop is indeed <http://localhost:8080>

In an initial pedagogical approach, two methods are useful:

- Proxy mode: ZAP intercepts requests sent by the browser or Postman.
- Direct scan mode: ZAP explores and scans the target directly.

For a first guided workshop, it is advisable to start with direct scanning on a simple local application, then introduce proxy interception if the teacher wants to show detailed traffic observation.

2.14 First exploratory scan with OWASP ZAP

Once DigiBank is started, launch a simple initial exploratory test in ZAP:

1. Choose Quick Start.
2. Select Automated Scan.
3. Enter the target URL: <http://localhost:8080>
4. Start the scan.

This operation allows ZAP to discover exposed resources, scan certain accessible routes, and generate initial passive or active alerts depending on the configuration. In the context of DigiBank, the initial scan must, at a minimum, reveal the application root, the homepage, Swagger UI, and a portion of the publicly accessible routes.

However, the document must explain that this initial scan is only a reconnaissance step. It is not sufficient, on its own, to draw conclusions about the application's security. Its primary function is to map the observable surface, identify test points, and demonstrate to the student how a DAST tool discovers an application from the outside.

2.15 Verification of HTTP interception via the ZAP proxy

To go beyond simple automated scanning, it is useful to demonstrate how ZAP can observe the actual requests sent to DigiBank. This step has significant educational value, as it makes the application's HTTP behavior visible and helps students understand what it truly means to "**dynamically analyze a system**".

Configure the browser or Postman to use ZAP's local proxy, usually:

- Host: **localhost**
- Port: **8080** or **8090** depending on the chosen local ZAP configuration

Then make a few simple calls to DigiBank:

- Opening <http://localhost:8080>
- Opening <http://localhost:8080/swagger-ui.html>
- GET call `/api/customers`
- GET call `/api/accounts`

The requests should then appear in the ZAP history. This allows you to:

- View HTTP headers
- Observe the response statuses
- Examine the JSON responses,
- Prepare for possible manual adjustments to parameters.

This step is particularly useful before later parts of the workshop, where participants will be asked to interpret the presence of error messages, exposed resources, or unexpected behavior.

2.16 Newman preparation for HTTP scenario replay

Although Postman is primarily an interactive tool, it can be integrated into more automated logic via Newman. This allows for the rapid replay of a collection of calls against DigiBank and verification that the basic behavior remains stable before or after certain manipulations.

Install Newman if Node.js is available:

```
npm install -g newman
```

Then run the exported collection:

```
newman run  
dast/postman/DigiBank-DAST-Collection.postman_collection.json \  
-e dast/postman/DigiBank-Local.postman_environment.json
```

This command is not yet an advanced DAST scan in itself, but it introduces an important element of DevSecOps culture: a basic HTTP scenario must be able to be replayed reproducibly, without relying solely on manual clicks in a graphical interface.

2.17 Recommended organization of DAST files in the repository

To maintain a clean and reusable structure, it is recommended to group the DAST elements in a dedicated directory at the project root. This approach respects the structuring logic already well established in workshops 1 and 2.

A simple and clear tree structure could be the following:

```
digibank-parent/  
  
  dast/  
  
    postman/  
  
      DigiBank-DAST-Collection.postman_collection.json  
  
      DigiBank-Local.postman_environment.json  
  
    zap/  
  
      zap-notes.md  
  
      scan-reports/
```

The zap-notes.md file can contain:

- The tested target URL
- The date of the scan
- The options used
- Remarks on discovered endpoints
- The first warning signs observed
- the known limitations of local scanning.

This discipline prepares for the rest of the workshop, where participants will be asked not only to launch tools, but also to interpret and document the results.

2.18 Important orders of this phase

Once the environment is prepared, the essential commands for this section can be summarized:

```
CD digibank-parent  
  
mvn clean install
```

```
mvn spring-boot:run -pl digibank-web

docker ps

docker logs digibank-postgres

docker compose up --build

docker compose ps

docker compose logs -f

newman run
dast/postman/DigiBank-DAST-Collection.postman_collection.json \
-e dast/postman/DigiBank-Local.postman_environment.json
```

Each one has a specific function:

- **mvn clean install** rebuilds and validates the project.
- **mvn spring-boot:run -pl digibank-web** starts DigiBank locally.
- **docker ps** and **docker logs** check the status of the database or containers.
- **docker compose up --build** launches the reproducible application environment.
- **newman run ...** replays the basic HTTP collection.

2.19 Final readiness check before DAST tests

At the end of this phase, the student must be able to demonstrate that the DAST environment is truly ready. The minimum criteria are as follows:

- The DigiBank project opens correctly in IntelliJ IDEA.
- The complete Maven build runs without any blocking errors.
- PostgreSQL is available locally or via Docker.
- The DigiBank application starts correctly.
- The homepage can be accessed at <http://localhost:8080>.
- Swagger UI can be accessed at <http://localhost:8080/swagger-ui.html>.
- The main endpoints respond via Swagger or Postman.

- A basic Postman collection exists and can be replayed.
- OWASP ZAP can reach the application and produce an initial map.

A general verification sequence can be given as follows:

```
CD digibank-parent

mvn clean install

docker compound up -d --build

newman run
dast/postman/DigiBank-DAST-Collection.postman_collection.json \
-e dast/postman/DigiBank-Local.postman_environment.json
```

Then check in the browser:

- <http://localhost:8080>
- <http://localhost:8080/swagger-ui.html>
- <http://localhost:8080/api-docs>

And finally, run a first quick scan in ZAP on <http://localhost:8080>.

2.20 Expected result at the end of part 2

At the end of Part 2, the student should not yet have conducted a detailed analysis of DigiBank's dynamic vulnerabilities, but they should have prepared all the necessary groundwork for this analysis. They have an executable application, a consistent database, an exploitable dataset, a reusable Postman collection, a reproducible Docker environment if needed, and a DAST tool such as OWASP ZAP ready to map and observe the application surface.

In other words, DigiBank here goes from the status of an application simply corrected after SAST to that of a system prepared for dynamic observation, which corresponds exactly to the pedagogical logic of the course: after learning to secure the code upstream, it becomes possible to verify how the application actually behaves when it runs and is exposed to external interactions.



Part 3: Detailed Dynamic Analysis of DigiBank, Observation of Real-World Behavior, and Implementation of DAST

3.1 Role of this phase

At this stage of Workshop 3, DigiBank should no longer be approached simply as a project to be read or fixed statically, but as a running application whose observable behavior itself constitutes an object of security analysis. This difference is fundamental. In Workshop 2, the student examined classes, DTOs, services, exceptions, and configuration files to identify structural weaknesses. In Workshop 3, they must now verify how these choices actually translate when an HTTP client interacts with the application, when an invalid request is sent, when an endpoint is explored, or when a dynamic scanner observes the exposed surface.

This section is central to the DAST pedagogical approach. The course emphasizes that certain weaknesses only become apparent at runtime: insufficient server-side

validation, overly verbose error messages, exposure of interactive documentation, overly rich JSON responses, or insufficiently hardened HTTP configuration. The goal here is therefore not to abstractly revisit DAST concepts, but to materialize them on DigiBank by demonstrating, for each important functional area, what to observe in the code, what to dynamically verify, which tools to use, and how to interpret the results.

3.2 Expected General Method

As in Part 3 of the SAST Lab 2, students should not simply launch a tool and copy a report. A thorough dynamic analysis relies on a structured sequence. First, it's necessary to start with a properly running application, with an accessible PostgreSQL database and a consistent Spring profile. Next, the code must identify which areas warrant priority dynamic observation. Only then does it become relevant to prepare queries in Postman, execute abnormal scenarios, observe HTTP responses, and then complete the analysis with OWASP ZAP and, finally, with Newman to automate revalidation.

The logic is therefore as follows:

- read the code to identify sensitive behaviors that may be visible at runtime;
- launch DigiBank in a stable environment;
- use Swagger to quickly map exposed endpoints;
- use Postman to run precise and reproducible tests;
- use ZAP to observe the attack surface, headers, passive alerts and certain automatically detectable behaviors;
- document the observations in the form of vulnerability reports;
- prepare the remediation in the next section.

3.3 Tools to use in this section

The tools in this section should be presented as complementary, just as SonarQube, Dependency-Check and PITest were in SAST Lab 2.

IntelliJ IDEA

IntelliJ IDEA is used to open DigiBank, read controllers, DTOs, services, exception handlers, and YAML files, and then run or modify the project. In a DAST workbench, it's not just used



for coding; it's also used to anticipate what will be dynamically observable based on choices already visible in the code.

Spring Boot

Spring Boot provides the DigiBank runtime framework. It allows the application to be launched locally and exposes HTTP routes which will then be tested by Postman, Swagger, curl, Newman and ZAP.

PostgreSQL

PostgreSQL plays a crucial role, as DigiBank's dynamic behaviors depend on the actual state of the data. An invalid transfer, a search for a non-existent identifier, or an attempt to create a duplicate account cannot be properly observed if the database is inaccessible or if the minimum test data is not present.

Swagger / OpenAPI

Swagger UI facilitates the discovery of endpoints, JSON schemas, and available operations. In this context, Swagger is not just a convenient testing tool; it also constitutes a testing surface itself to be analyzed from a DAST perspective.

Postman

Postman is the primary tool for building and replaying scenarios. It allows you to prepare requests, organize test cases by collection, precisely observe HTTP responses, and maintain a clear record of normal and abnormal scenarios.

Newman

Newman allows you to automatically execute a Postman collection from the command line. It is very useful at the end of remediation, as it allows you to transform validated manual scenarios into a reproducible revalidation sequence.

OWASP ZAP

ZAP is the main DAST tool of the workshop. It allows you to explore DigiBank, automatically discover certain resources, observe HTTP responses, identify passive alerts and highlight elements such as the exposure of documentation, the absence of certain headers or revealing responses.

curl

curl is not the central tool, but it plays an important pedagogical role. It allows us to demonstrate that DAST tests do not depend on a graphical interface, and that any exposed HTTP route can be queried directly from the command line.

3.4 Practical preparation of the DAST environment

Before proceeding with the detailed analysis, it is essential to ensure that DigiBank starts cleanly. The student should begin with a stable version of the project, ideally already statically hardened in workshop 2, so that workshop 3 observes a credible foundation and not an arbitrarily broken application.

3.4.1 Spring configuration verification

Create or open the file:

digibank-web/src/main/resources/application-dev.yml

Points to check in this file:

- PostgreSQL datasource URL;
- username and password passed via environment variables or local configuration;
- server port;
- possible activation of Swagger;
- log verbosity level;
- presence or absence of show-sql;
- exposed springdoc paths.

Example of a starting configuration compatible with DAST work:

```
server:  
  
  port: 8080  
  
spring:
```

```
datasource:

    url: ${ SPRING_DATASOURCE_URL : jdbc : postgresql ://
localhost : 5432 / digibankdb }

    username: ${ SPRING_DATASOURCE_USERNAME : digibank }

    password: ${ SPRING_DATASOURCE_PASSWORD : digibank123 }

jpa:

    hibernate:

        ddl-auto: update

        show-sql: false

springdoc:

    api-docs:

        path: /api-docs

    swagger-ui:

        path: /swagger-ui.html
```

Observation to be made based on the code and configuration:

- Is Swagger enabled without restriction?
- Are the documentation routes directly accessible?
- Are the logging parameters likely to make the responses or behavior too verbose?
- Is the application based on a clearly identified profile for local development?

3.4.2 Starting the application

Order type:

```
export SPRING_DATASOURCE_URL = jdbc : postgresql :// localhost :  
5432 / digibankdb  
  
export SPRING_DATASOURCE_USERNAME = digibank  
  
export SPRING_DATASOURCE_PASSWORD = digibank123  
  
mvn clean verify  
  
mvn spring-boot:run -pl digibank-web
```

To observe:

- The application starts without error;
- The Spring context is initialized;
- REST routes are exposed;
- Swagger UI is accessible;
- No abnormal stack traces appear at startup.

3.5 Code Reading Before DAST Tests

As in Part 3 of SAST Lab 2, it is essential to first read the code to know what to look for during execution. DAST does not replace this reading; it extends it.

3.5.1 GlobalExceptionHandler

Create or open the file:

digibank-web/src/main/java/com/m2ibank/web/exception/GlobalExceptionHandler.java

Observation to be made based on the code:

- Are the `ResourceNotFoundException`, `BusinessException` and `MethodArgumentNotValidException` errors transformed into concise or overly detailed responses?

- Does the handler concatenate the field names and validation messages?
- Are generic errors being properly masked?

Why this reading prepares you for the DAST:

If the handler directly returns `ex.getMessage()`, there is a high chance that Postman and ZAP will observe responses that reveal too precisely the internal state of the system or the nature of the failed validation.

3.5.2 CustomerRequest

Create or open the file:

`customer-module/src/main/java/com/m2ibank/customer/dto/CustomerRequest.java`

Observation to be made based on the code:

- `@NotBlank`, `@Size` and `@Email` are present, but do `phoneNumber` and `nationalId` have a real format constraint?
- Does the absence of `@Pattern` allow inconsistent values to pass through?

Why this reading prepares you for the DAST:

If server-side validation is incomplete, Postman may often allow abnormal entries to be accepted or obtain revealing rejection messages.

3.5.3 Customer Response

Create or open the file:

`customer-module/src/main/java/com/m2ibank/customer/dto/CustomerResponse.java`

Observation to be made based on the code:

- Does `nationalId` appear in the standard response?
- Is `phoneNumber` fully exposed?
- Does the response provide more information than necessary?

Why this reading prepares you for the DAST:

If the output DTO is too rich, a simple GET /api/customers will make it immediately visible at runtime.

3.5.4 Customer Service

Create or open the file:

customer-module/src/main/java/com/m2ibank/customer/service/CustomerService.java

Excerpts to reread first:

- createCustomer(...)
- getCustomerById(...)

Observations to be made based on the code:

- Do A customer with this email already exists or Customer not found with id X messages flow up to the REST layer?
- Is the identifier or sensitive business data injected into the error message?

Why this reading prepares you for the DAST:

These messages can be observed directly in Postman when duplicate creations or readings of non-existent identifiers.

3.5.5 AccountRequest and AccountService

Create or open:

- account-module/src/main/java/com/m2ibank/account/dto/AccountRequest.java
- account-module/src/main/java/com/m2ibank/account/service/AccountService.java

Observations to be made based on the code:

- Is initialBalance only controlled by @DecimalMin?
- Does the service actually revalidate critical assumptions?
- Does the message "Account not found with id X" still appear?

Why this reading prepares you for the DAST:

Scenarios involving a non-existent account, a negative initial balance, or reading an absent account may confirm these weaknesses in execution.

3.5.6 TransferRequest and TransferService

Create or open:

- transfer-module/src/main/java/com/m2ibank/transfer/dto/TransferRequest.java
- transfer-module/src/main/java/com/m2ibank/transfer/service/TransferService.java

Observations to be made based on the code:

- Does the DTO only enforce @DecimalMin(0.01)?
- Does the service explicitly verify that the accounts are separate?
- Does the service protect zero or negative amounts even if the DTO were to be bypassed?
- Do rejection messages reveal too much about the business state, such as "Insufficient balance for transfer"?

Why this reading prepares you for the DAST:

Transfer routes are the most sensitive; they often give rise to the most telling DAST observations on business validation, errors, and information exposure.

3.6 Implementing dynamic tests with Postman

3.6.1 Creating the Postman environment

Create a Postman environment named DigiBank Local with at least the following variable:

- baseUrl = http://localhost:8080

Next, create a DigiBank DAST collection with the following folders:

- Customers
- Accounts
- Transfers

- Discovery
- Negative tests

This organization is pedagogically useful because it allows us to distinguish between mapping tests, nominal tests, and attack or negative validation tests.

3.6.2 Initial Mapping

Create the following queries in Postman:

```
GET {{baseUrl}}/  
  
GET {{baseUrl}}/swagger-ui.html  
  
GET {{baseUrl}}/api-docs  
  
GET {{baseUrl}}/api/customers  
  
GET {{baseUrl}}/api/accounts
```

Observations to be recorded:

- which endpoints are immediately accessible;
- which endpoints return 200;
- if Swagger UI and /api-docs are freely exposed;
- if the JSON responses contain sensitive or overly detailed data.

3.6.3 Customer Testing

Nominal request

```
POST {{baseUrl}}/api/customers  
  
Content-Type: application/json
```

```
{  
  
  "fullName" : "Jean Testeur",  
  
  "email" : "jean.testeur@digibank.com",  
  
  "phoneNumber" : "237699999999"  
  
  "nationalId" : "CNI999999"  
  
}
```

Observations:

- HTTP code;
- structure of the response;
- fields exposed in data or in the JSON body;
- possible presence of nationalId in plain text.

Invalid email test

```
{  
  
  "fullName" : "Invalid Client",  
  
  "email" : "invalid email",  
  
  "phoneNumber" : "2376999999998",  
  
  "nationalId" : "CNI9999998"  
  
}
```

Observations:

- Is the request refused?
- Is the HTTP code consistent?
- Is the error message too specific?

- Is the exact name of the field displayed?
- Does the validation structure seem too verbose?

Duplicate email test

First create a valid client, then replay the same request with the same email.

Expected observation:

- if the response contains "A customer with this email already exists", the application explicitly reveals the existence of the data;
- This observation should be compared with the code of `CustomerService.createCustomer(...)`.

Non-existent identifier test

```
GET {{baseUrl}}/api/customers/99999
```

Expected observation:

- If the response contains "Customer not found with id 99999", this confirms that the identifier provided by the customer is used as is in the public message.

3.6.4 Tests on accounts

Nominal creation

```
POST {{baseUrl}}/api/accounts  
  
Content-Type: application/json
```

```
{  
  
  "customerId" : 1,  
  
  "accountType" : "CURRENT",
```

```
"initialBalance" : 5000  
}
```

Observations:

- structure of the response;
- financial data exposed;
- possible presence of excessive information.

Initial negative balance

```
{  
  
  "customerId" : 1,  
  
  "accountType" : "CURRENT",  
  
  "initialBalance" : -5000  
}
```

Observations:

- Is the request rejected?
- Is the rejection due to a declarative validation or a business logic?
- Does the message contain too many technical details?

Non-existent account

```
GET {{baseUrl}}/api/accounts/99999
```

Expected observation:

- Possible presence of the message "Account not found with id 99999";
- dynamic confirmation of a pattern already statically visible in `AccountService.getAccountEntityById(...)`.

3.6.5 Tests on wire transfers

Nominal transfer

```
POST {{baseUrl}}/api/transfers
Content-Type: application/json
```

```
{
  "sourceAccountId" : 1,
  "destinationAccountId" : 2,
  "amount" : 1000,
  "description" : "Test transfer"
}
```

Observations:

- functional success;
- structure of the response;
- exposed business data.

Transfer to the same account

```
{  
  
  "sourceAccountId" : 1,  
  
  "destinationAccountId" : 1,  
  
  "amount" : 1000,  
  
  "description" : "Same account"  
}
```

Expected observation:

- rejection with business message;
- need to assess whether the message is pedagogically useful but too verbose regarding safety.

Zero amount

```
{  
  
  "sourceAccountId" : 1,  
  
  "destinationAccountId" : 2,  
  
  "amount" : 0,  
  
  "description" : "Zero amount"  
}
```

Expected observation:

- if the DTO is sufficient on its own, the validation can reject the case;
- if the service is not defensive, some borderline scenarios may remain fragile;

- This test directly prepares the remediation on the service side.

Insufficient balance

```
{  
  
  "sourceAccountId" : 1,  
  
  "destinationAccountId" : 2,  
  
  "amount" : 999999999,  
  
  "description" : "Insufficient balance"  
  
}
```

Expected observation:

- message of the type Insufficient balance for transfer;
- confirmation that the application explicitly reveals a business status of the account.

3.7 Using OWASP ZAP

3.7.1 Installation and startup

The student must launch ZAP and then use either direct browsing of <http://localhost:8080> or proxy mode if Postman or browser requests are to be routed through ZAP. In an educational setting, the simplest approach is often to start with a passive exploration of the local site and observe the resulting directory structure.

3.7.2 What to observe in ZAP

In ZAP, you must note at a minimum:

- routes discovered automatically;
- the presence of `/swagger-ui.html` and `/api-docs`;

- the visible REST endpoints;
- passive alerts on HTTP headers;
- the answers containing revealing messages;
- the parameters identified in the queries.

3.7.3 Link with the code

ZAP alerts or observations should always be linked to a visible application choice in the code or configuration:

- Swagger exposure linked to springdoc in application-dev.yml;
- overly explicit error message related to GlobalExceptionHandler or services;
- Insufficient validation related to DTOs and services;
- Overly rich responses related to output DTOs.

3.8 Use of curl for some targeted tests

To show that the tests are reproducible outside of a graphical interface, a few simple commands can be provided.

Example of reading a non-existent customer:

```
curl -i http://localhost:8080/api/customers/99999
```

Example of an invalid customer creation:

```
curl -X POST http://localhost:8080/api/customers \
-H "Content-Type: application/json" \
-d '{ "fullName": "", "email": "bad-email", "phoneNumber": "abc123", "nationalId": "???" } '
```

Example of an invalid transfer:

```
curl -X POST http://localhost:8080/api/transfers \
-H "Content-Type: application/json" \
-d '{ "sourceAccountId":1, "destinationAccountId":1, "amount":0, "description": "invalid" } '
```

These commands serve as minimal demonstration support and also prepare for possible simpler automation in a pipeline or verification script.

3.9 Expected feedback sheet for each dynamic vulnerability

As in SAST Lab 2, the report should not be a simple compilation of screenshots. For each weakness observed dynamically, the student must provide a structured report including:

- module or endpoint concerned;
- file or code area that was the likely source;
- request executed;
- answer obtained;
- type of weakness observed;
- potential impact;
- tool used;
- anticipated correction.

Example :

- Endpoint: GET /api/customers/99999
- Affected code: CustomerService.getCustomerById(...)
- Observation: Customer not found with id 99999 message
- Weakness: overly revealing error message
- Impact: Facilitates the enumeration of identifiers

- Tool: Postman / curl
- Proposed correction: more generic public message, details reserved for logs.

Part 4: Detailed remediation of dynamic vulnerabilities, DigiBank hardening, and complete revalidation

4.1 Role of this phase

As in Part 4 of SAST Lab 2, the objective is no longer simply to observe signals or describe vulnerabilities. It is now about rationally correcting the dynamically confirmed weaknesses without breaking DigiBank's business behavior or introducing new technical debt. This part is the operational core of Workshop 3, as it transforms the DAST findings into changes to code, DTOs, error handling, configuration, and test strategy.

Remediation should follow the same logic as that already adopted in TP2 SAST: initial vulnerable code, corrected code, explanation of the correction, verification commands or tests, then expected impact on tools and observable behaviors.

4.2 Remediation of insufficient entry validations on clients

Vulnerable initial code

Create or open:

customer-module/src/main/java/com/m2ibank/customer/dto/CustomerRequest.java

Typical problematic version:

```
audience class CustomerRequest {
```

```

@NotBlank ( message ) = "Full name is required "

@ Size ( max = 120 )

    private String fullName ;


@NotBlank ( message ) = "Email is required "

@ Email ( message ) = "Email format is invalid "

    private String email ;


@NotBlank ( message ) = "Phone number is required "

@ Size ( max = 30 )

    private String phoneNumber ;


@NotBlank ( message ) = "National ID is required "

@ Size ( max = 30 )

    private String nationalId ;

}

```

Issue :

The presence of @Size gives an impression of security, but neither phoneNumber nor nationalId are actually constrained in their structure. This weakness, already visible statically, becomes dynamically confirmable when Postman accepts or attempts to accept absurd values like abc123 or !!!.

Code corrected

```

audience class CustomerRequest {

    @NotBlank ( message ) = "Full name is required "

    @ Size ( max = 120 , message = "Full name must not exceed 120
characters" )

        private String fullName ;

    @NotBlank ( message ) = "Email is required "

    @ Email ( message) = "Email format is invalid "

    @ Size ( max = 120 , message = "Email must not exceed 120
characters" )

        private String email ;

    @NotBlank ( message ) = "Phone number is required "

    @Pattern (

        regexp = "[0-9]{9,15}$" ,

        message = "Phone number must contain only digits and have
between 9 and 15 characters"

    )

        private String phoneNumber ;

    @NotBlank ( message ) = "National ID is required "

```

```

@Pattern (

    regexp = "[A-Z0-9 \\ -]{5,30}$" ,

    message = "National ID format is invalid"

)

private String nationalId ;

    audience void setEmail ( String email ) {

        this . email = email == null ? null : email . trim ().
toLowerCase ();

    }

    audience void setPhoneNumber ( String phoneNumber ) {

        this . phoneNumber = phoneNumber == null ? null :
phoneNumber.trim ( ) ;

    }

    audience void setNationalId ( String nationalId ) {

        this . nationalId = nationalId == null ? null :
nationalId . trim (). toUpperCase ();

    }

}

```

Explanation of the correction:

Remediation strengthens server-side validation with real form constraints and adds minimal normalization. This reduces business inconsistencies and improves the quality of recorded data. In a DAST context, the benefit is immediate: malformed requests sent from Postman should now be consistently rejected.

Verification tests:

```
curl -X POST http://localhost:8080/api/customers \
-H "Content-Type: application/json" \
-d '{ "fullName": "Test", "email": "bad-email", "phoneNumber": "abc123", "nationalId": "???" } '
```

Expected result:

- HTTP 400;
- consistent validation message;
- No stack trace or unnecessary technical details.

4.3 Remediation of overly rich API responses on clients

Vulnerable initial code

Create or open:

customer-module/src/main/java/com/m2ibank/customer/dto/CustomerResponse.java

Typical problematic version:

```
audience class CustomerResponse {

    private Long id ;

    private String fullName ;
```

```
private String email ;

private String phoneNumber ;

private String nationalId ;

private LocalDateTime createdAt ;

}
```

Issue :

The nationalId and sometimes the full phone number are exposed in the standard response. This violates the principle of data minimization and makes the vulnerability directly observable with a simple GET /api/customers.

Code corrected

```
audience class CustomerResponse {

    private Long id ;

    private String fullName ;

    private String email ;

    private String phoneNumber ;

    private LocalDateTime createdAt ;

}
```

An even cleaner option:

```
audience class CustomerSummaryResponse {

    private Long id ;
```

```
private String fullName ;

private String email ;

private LocalDateTime createdAt ;

}
```

Explanation of the correction:

The API response should contain only what is necessary for the client's normal use of the API. Removing nationalId directly reduces the dynamically observable information leakage.

Verification test:

```
GET {{baseUrl}}/api/customers
```

Expected result:

- the response no longer includes nationalId;
- The data presented is more understated.

4.4 Remediation of overly revealing error messages

Vulnerable initial code

Create or open:

customer-module/src/main/java/com/m2ibank/customer/service/CustomerService.java

Problematic examples:

```
throw new BusinessException ( "A customer with this email  
already exists" );
```

```
throw new BusinessException ( "A customer with this phone number  
already exists" );  
  
throw new ResourceNotFoundException ( "Customer not found with  
id " + id);
```

Create or open as well:

account-module/src/main/java/com/m2ibank/account/service/AccountService.java

Problematic examples:

```
throw new ResourceNotFoundException ( "Account not found with id  
" + id);  
  
throw new BusinessException ( "Insufficient balance for  
transfer" );
```

Issue :

These messages are functionally useful, but too revealing from a security perspective. They confirm the existence of resources, the presence of data in the database, or the business status of an account.

```
throw new BusinessException ( "Request could not be completed"  
);  
  
throw new ResourceNotFoundException ( "Customer not found" );  
  
throw new ResourceNotFoundException ( "Account not found" );  
  
throw new BusinessException ( "Operation not allowed" );
```

But the best fix is to couple this with a cleaner global handler.

Code corrected on the global manager side

Create or replace:

common-module/src/main/java/com/m2ibank/common/api/ApiError.java

```
package com.m2ibank.common.api ;

import java.time.LocalDateTime ;

import java.util.List ;

audience class ApiError {

    private boolean success ;

    private String message ;

    private List < String > details ;

    private LocalDateTime timestamp ;

    audience ApiError () {

        this.success = false ;

        this.timestamp = LocalDateTime.now ( ) ;

    }

    audience ApiError ( String message , List < String > details
) {

        this ( ) ;
    }
}
```

```
        this . message = message;

        this . details = details;
    }

    audience static ApiError of ( String message , List < String
> details ) {

        return new ApiError (message, details);
    }

    audience boolean isSuccess () {

        return success;
    }

    audience String getMessage () {

        return message;
    }

    audience List <String> getDetails ( ) {

        return details;
    }

    audience LocalDateTime getTimestamp () {
```

```
        return timestamp;
    }
}
```

Then replace the handler:

digibank-web/src/main/java/com/m2ibank/web/exception/GlobalExceptionHandler.java

```
package com.m2ibank.web.exception ;

import com.m2ibank.common.api.ApiError ;

import com.m2ibank.common.exception.BusinessException ;

import com.m2ibank.common.exception.ResourceNotFoundException ;

import org.springframework.http.HttpStatus ;

import org.springframework.http.ResponseEntity ;

import
org.springframework.web.bind.MethodArgumentNotValidException ;

import org.springframework.web.bind.annotation.ExceptionHandler
;

import
org.springframework.web.bind.annotation.RestControllerAdvice ;

import java.util.List ;

@RestControllerAdvice
```

```
audience class GlobalExceptionHandler {

    @ExceptionHandler ( ResourceNotFoundException.class )

        audience ResponseEntity <ApiError> handleNotFound (
ResourceNotFoundException ex ) {

            return ResponseEntity . status ( HttpStatus . NOT_FOUND )

. body ( ApiError . of ( "Resource not found" , List . of ( "The
requested resource does not exist" ))) ;

        }

    @ExceptionHandler ( BusinessException.class )

        audience ResponseEntity <ApiError> handleBusinessException (
BusinessException ex ) {

            return ResponseEntity . status ( HttpStatus.BAD_REQUEST )

. body ( ApiError . of ( "Request could not be processed" , List
. of ( "Business rule validation failed" ))) ;

        }

    @ExceptionHandler ( MethodArgumentNotValidException.class )

        audience ResponseEntity <ApiError> handleValidationException
( MethodArgumentNotValidException ex ) {

            return ResponseEntity . status ( HttpStatus.BAD_REQUEST )
```

```

    . body ( ApiError . of ( "Input validation failed" , List . of (
    "One or more fields are invalid" ))) ;

}

@ExceptionHandler ( Exception.class )

    audience ResponseEntity <ApiError> handleGenericException (
Exception ex ) {

        return ResponseEntity . status ( HttpStatus .
INTERNAL_SERVER_ERROR )

        . body ( ApiError . of ( "Internal server error" , List . of (
"The operation could not be completed" ))) ;

}

}

```

Explanation of the correction:

The response structure becomes more stable and streamlined. The precise details needed for diagnosis can remain in internal logs but are no longer exposed directly to the HTTP client. This fix is one of the most noticeable in DAST, as it immediately transforms the responses observed in Postman and ZAP.

Verification tests:

```

GET {{baseUrl}}/api/customers/99999

POST {{baseUrl}}/api/customers    // e-mail Already existing

```

```
POST {{baseUrl}}/api/transfers // account source = account
destination
```

Expected results:

- more generic messages;
- absence of raw identifiers in the errors;
- homogeneous error structure.

4.5 Remediation of insufficiently enforced business rules in transfers

Vulnerable initial code

Create or open:

transfer-module/src/main/java/com/m2ibank/transfer/service/TransferService.java

Typical problematic version:

```
@ Transactional

audience TransferResponse createTransfer ( TransferRequest
request) {

    if ( request.getSourceAccountId ( ). equals (
request.getDestinationAccountId ( ) )) {

        throw new BusinessException ( "Source and destination
accounts must be different" );

    }

    Account sourceAccount = accountService . getAccountEntityById
( request.getSourceAccountId ( ) );
```

```

        Account destinationAccount = accountService .
getAccountEntityById ( request.getDestinationAccountId ( ) );

        accountService . debitAccount ( sourceAccount,
request.getAmount ( ) );

        accountService . creditAccount ( destinationAccount,
request.getAmount ( ) );

        Transfer transfer = new Transfer (

            request . getSourceAccountId ( ),

            request . getDestinationAccountId ( ),

            request.getAmount ( ) ,

            request.getDescription ( )

        );

        Transfer savedTransfer = transferRepository . save
(transfer);

        return mapToResponse (savedTransfer);
    }

```

Issue :

The main business rule exists, but the service remains too dependent on the DTO for certain critical validations. However, from a defensive standpoint, a financial operation must also express its own safeguards at the service level.

Code corrected

```
@ Transactional

audience TransferResponse createTransfer ( TransferRequest
request) {

    if (request == null ) {

        throw new BusinessException ( "Request could not be
processed" );

    }

    if ( request.getSourceAccountId ( ) == null || request .
getDestinationAccountId ( ) == null ) {

        throw new BusinessException ( "Request could not be
processed" );

    }

    if ( request.getAmount ( ) == null || request . getAmount ( ) .
signum ( ) <= 0 ) {

        throw new BusinessException ( "Request could not be
processed" );

    }
}
```

```

        if ( request.getSourceAccountId ( ). equals (
request.getDestinationAccountId ( ) )) {

            throw new BusinessException ( "Request could not be
processed" );

        }

        String description = request . getDescription ( ) == null ? "
: request . getDescription ( ). trim ( );

        Account sourceAccount = accountService . getAccountEntityById
( request.getSourceAccountId ( ) );

        Account destinationAccount = accountService .
getAccountEntityById ( request.getDestinationAccountId ( ) );

        accountService . debitAccount ( sourceAccount,
request.getAmount ( ) );

        accountService . creditAccount ( destinationAccount,
request.getAmount ( ) );

        Transfer transfer = new Transfer (

            request . getSourceAccountId ( ),

            request . getDestinationAccountId ( ),

            request.getAmount ( ) ,

```

```

description

);

        Transfer savedTransfer = transferRepository . save
(transfer);

        return mapToResponse (savedTransfer);
}

```

Explanation of the correction:

The service no longer blindly trusts the DTO. It explicitly protects critical business prerequisites. This approach is particularly important for DAST, as it ensures that even indirect calls or edge cases produce consistent rejections.

Verification tests:

```

curl -X POST http://localhost:8080/api/transfers \

-H "Content-Type: application/json" \

-d '{ "sourceAccountId":1,"destinationAccountId":1,"amount":1000,"de
scription":"same"}'

curl -X POST http://localhost:8080/api/transfers \

-H "Content-Type: application/json" \

```

```
-d
'{"sourceAccountId":1,"destinationAccountId":2,"amount":0,"description":"zero"}'
```

Expected results:

- HTTP 400;
- coherent message;
- no unnecessary exposure of the internal state.

4.6 Remediation of debit rules in AccountService

Vulnerable initial code

Create or open:

account-module/src/main/java/com/m2ibank/account/service/AccountService.java

Typical problematic version:

```
audience void debitAccount ( Account account, BigDecimal amount)
{
    if ( account . getBalance (). compareTo (amount) < 0 ) {
        throw new BusinessException ( "Insufficient balance for
transfer" );
    }

    account . setBalance ( account.getBalance ( ) . subtract
(amount));

    accountRepository . save (account);
}
```

Issue :

The method protects against the case of insufficient balance, but remains too verbose and insufficiently defensive against null or zero or negative amounts.

Code corrected

```
audience void debitAccount ( Account account, BigDecimal amount)
{
    if (account == null ) {
        throw new BusinessException ( "Operation not allowed" );
    }

    if (amount == null || amount.signum ( ) <= 0 ) {
        throw new BusinessException ( "Operation not allowed" );
    }

    if ( account . getBalance ( ). compareTo (amount) < 0 ) {
        throw new BusinessException ( "Operation not allowed" );
    }

    account . setBalance ( account.getBalance ( ) . subtract
(amount));

    accountRepository . save (account);
}
```

Explanation of the correction:

The service is hardened and becomes more consistent with a conservative error management strategy. This improves both business robustness and the quality of dynamically observable responses.

4.7 Remediation of Swagger Exposure According to the Environment

Vulnerable initial code

Create or open:

digibank-web/src/main/resources/application-dev.yml

Typical version:

```
springdoc :  
  
  api-docs :  
  
    path : /api-docs  
  
  swagger-ui :  
  
    path : /swagger-ui.html
```

Issue :

Interactive documentation is displayed directly. This is acceptable for educational development, but it should be treated as a conscious choice and not as a permanent display.

Suggested correction

Keep Swagger in development, but disable it or control it in other profiles.

Example application-dev.yml:

```
springdoc :  
  
  api-docs :
```

```
enabled : true

path : /api-docs

swagger-ui :

  enabled : true

  path : /swagger-ui.html
```

Example application-prod.yml:

```
springdoc :

  api-docs :

    enabled : false

  swagger-ui :

    enabled : false
```

Explanation of the correction:

Remediation doesn't necessarily mean eliminating Swagger everywhere. It means making it dependent on the environment. DAST (Digital Access to Science and Technology) should teach students that a useful exposed surface area during development can become unnecessarily dangerous elsewhere.

Verification test:

- launch with the dev profile, verify that Swagger remains accessible;
- Launch with the prod profile, verify that /swagger-ui.html and /api-docs are no longer exposed.

4.8 Revalidation with Postman and Newman

After correction, the Postman collections built in part 3 must be replayed. The following must be checked:

- that the nominal cases always work;
- that invalid cases are now rejected;
- that the messages are more understated;
- that the sensitive responses are less rich.

Export of the Postman collection, then execution with Newman:

```
newman run DigiBank-DAST.postman_collection.json \  
-e DigiBank-Local.postman_environment.json
```

Educational value:

Newman transforms manual scenarios into reproducible revalidation campaigns. He perfectly embodies the DevSecOps logic of repeatability and progressive automation.

4.9 Revalidation with ZAP

After remediation, the ZAP exploration must be restarted and the results compared:

- the presence or absence of Swagger depending on the profile;
- the reduction of information revealed by error messages;
- any alerts about HTTP responses;
- the overall evolution of the visible surface.

The student must learn to compare a before and after. The DAST (Diagnostic Assessment of System Performance) only has full educational value if it allows observation that the correction has actually changed the system's behavior.

4.10 Expected result at the end of parts 3 and 4

At the end of these two parts, the student should no longer be satisfied with simply knowing the definition of DAST. They should be able to start from DigiBank's code, anticipate dynamically observable risk areas, build consistent test scenarios with Postman, supplement the observation with ZAP, link the findings to DTOs, services, handlers and configuration files, then write the necessary corrections before revalidating everything.

In other words, it's essential to understand that dynamic analysis isn't an activity isolated from code choices. On the contrary, it's a natural extension of reading the project. The code prepares the observable behavior, DAST confirms this behavior at runtime, and remediation transforms this understanding into concrete improvements for DigiBank. It's precisely this interplay between code reading, execution testing, correction, and revalidation that gives Workshop 3 its full coherence within the DevSecOps progression of the course.

Part 5: Final validation, capitalization of DAST results and preparation for the next stage of the process

5.1 Role of final validation in workshop 3

After the dynamic observation of DigiBank, the execution of DAST scenarios, the interpretation of alerts, and the implementation of remediation measures, Workshop 3 cannot end simply because “the tests seem better.” The DAST methodology, as presented in the course, actually mandates a final validation step, because the objective is not merely to execute simulated attacks or run scanners, but to demonstrate that the exposed application now behaves more securely in real-world situations. In keeping with the learning path, this section therefore plays the same role as the final validation of Workshop 2: it transforms a series of technical actions into a demonstrable, justified, and usable result.

This phase is important for a simple reason. In DAST, a fix is never fully credible until it has been retested externally, under the same observation conditions that revealed the



initial weakness. A thorough final validation must therefore demonstrate that DigiBank remains functional, that the previously observed risky behaviors have been reduced or eliminated, that the testing tools no longer detect the same weak signals, and that the evidence produced is sufficiently structured to prepare for subsequent workshops, particularly those focused on dependencies, containers, and security industrialization.

5.2 What needs to be validated after remediation

As with a well-run SAST workshop, the end of a DAST workshop isn't simply about saying a fix has been applied. Several additional dimensions must be verified. The first concerns DigiBank's functional stability. The application must still start cleanly, respond to the main endpoints, accept valid requests, and continue to fulfill normal business use cases. A "hardened" API that has become unusable or inconsistent would not meet the course's pedagogical objective, which is to integrate security without breaking business value.

The second dimension concerns the actual reduction of dynamic signals observed before the fix. If Swagger was freely exposed and is no longer, if overly verbose error messages have been replaced with more concise responses, if server validation now rejects previously accepted abnormal input, or if security headers finally appear in HTTP responses, then the remediation has an observable and verifiable effect. Conversely, if the same behaviors remain visible from Postman or ZAP, then the fix is either incomplete or has not addressed the root cause of the problem.

The third dimension concerns the quality of the evidence. In a DAST workshop, results must be presented in a usable format: screenshots of Swagger and ZAP, Postman exports, Newman outputs, excerpts of before/after HTTP responses, and a reasoned summary linking the dynamic observation to the implemented correction. This requirement is consistent with the DevSecOps approach advocated in the course: security is not a feeling, it is documented, replayed, and demonstrated.

5.3 Expected technical checks

Final validation must be based on an explicit sequence of checks. The student must start with a remediated version of DigiBank, properly relaunch the application, and ensure that the API is accessible again. Specifically, this involves verifying that the Spring Boot backend starts without errors, that the connection to PostgreSQL remains functional, that

the OpenAPI documentation responds as expected if maintained in the test environment, and that the main business endpoints remain reachable.

An initial series of checks must be performed using the browser and Swagger. The objective here is no longer the initial discovery of the API, but verification after correction. The student should observe, based on the remediation applied, that:

- Access to Swagger may be limited to the development environment;
- some routes no longer return excessive information;
- the input schemes show stricter constraints;
- The error responses are more concise and consistent.

A second series of checks must be carried out using Postman. This is the main tool for fine-tuning the scenarios built in Part 3. The reference queries used during the dynamic analysis must be reused:

- valid queries to verify that normal behavior is intact;
- invalid requests for testing server validations;
- queries relating to non-existent identifiers;
- queries with zero, negative or inconsistent amounts;
- queries designed to cause application errors.

For each exercise, the student must compare the answers before and after correction. What is expected is not just a different HTTP code, but an interpretation of this change. For example:

- A 400 error with an overly detailed business message can become a 400 error with a more generic message;
- A 404 revealing the requested identifier can become a standardized 404;
- An abnormal request that was previously accepted can now be rejected;
- An overly complex response can be replaced by a more controlled DTO.

A third series of checks must be performed using OWASP ZAP. This time, the objective is not only to detect alerts but also to monitor their evolution. The student must run another targeted scan on the corrected version of DigiBank and then compare the results with those of the initial analysis phase. Specifically, it is necessary to observe whether:

- alerts related to information exposure are decreasing;
- Remarks on HTTP headers are reduced;
- certain unnecessary surfaces are no longer visible;

- The application's responses offer fewer technical clues to external tools.

This comparison must remain intelligent. As the course reminds us, DAST is not infallible, and some alerts may stem from the environment, scanner limitations, or pedagogical technical choices. The important thing, therefore, is not to produce an empty report at all costs, but to demonstrate that the vulnerabilities targeted in the workshop have been properly addressed and that the remaining signals have been understood.

5.4 Reuse of tools for final validation

The same logic of tools as in the previous parts must be retained, but here with a purpose of proof and revalidation.

a) Swagger UI

Swagger is used to quickly check the still-visible API surface and overall functional consistency. It allows you to:

- confirm that the endpoints are still available;
- verify that the query patterns make sense after hardening;
- observe if the documentation is still displayed in the correct environment;
- quickly replay a few simple calls.

However, students should not limit themselves to Swagger, as this tool is insufficient for documenting a complete DAST validation. It remains useful as a starting point for visual and functional verification.

b) Postman

Postman remains the primary tool for manual verification. It should be used to build a structured final validation collection, with folders such as:

- 01-functional-regression-check
- 02-input-validation-check
- 03-error-handling-check
- 04-information-exposure-check

- 05-remediation-verification

Each query must be clearly named and accompanied, in its description, by the test objective. Examples:

- Create customer - valid payload
- Create customer - invalid phone format
- Get customer - nonexistent id
- Transfer - negative amount rejected
- Transfer - same source and destination rejected

The student must also complete simple Postman tests to automatically verify certain elements. Examples of scripts:

```
pm.test ( " The HTTP status is 400" , function () {

pm.response.to.have. status ( 400 );

} );


pm.test ( " The response does not disclose a stack trace" ,
function ( ) {

p.m. expect (pm.response.text ( ). toLowerCase ( )).to.not.
include ( "exception" );

p.m. expect (pm.response.text ( ). toLowerCase ( )).to.not.
include ( "stack" );

p.m. expect (pm.response.text ( ). toLowerCase ( )).to.not.
include ( "trace" );

} );


pm.test ( " The response contains a checked message" , function
( ) {
```

```
const body = pm.response.json ( );

p.m. expect ( body ).to.have. property ( "message" );

} );
```

These tests do not replace human analysis, but they make validation more reproducible.

c) Newman

Newman should be used to automate the final validation. This is where its pedagogical value becomes clear. Once the Postman collection is stable, the student must run it automatically to demonstrate that the observed safe behaviors can be reproduced without manual intervention. A typical command might look like this:

```
newman run DigiBank-DAST-Validation.postman_collection.json \

-e DigiBank-local.postman_environment.json \

--reporters cli, html, json \

--reporter-html-export newman-report.html \

--reporter-json-export newman-report.json
```

What is expected of Newman is not just an “overall success”, but an interpretation of the report:

- how many tests were run;
- how many succeeded;
- what security scenarios were covered;
- if a remediation test fails again.

The Newman report thus becomes a piece of evidence of non-regression in security.

d) OWASP ZAP

ZAP should be used for a before/after comparison. The student can proceed in two ways:

- either replay a targeted scan on important roads;
- either use ZAP as a proxy during Postman or Swagger tests to observe exchanges and headers.

Examples of points to check in ZAP after correction include:

- reduction of visible technical messages;
- presence or improvement of security headers;
- decrease in observations related to the overly permissive configuration;
- disappearance of certain roads that were unnecessarily exposed.

The educational value of ZAP is strong here: it shows that the corrected application should not only “look safer in the code”, but be less verbose and less permissive from the outside, which corresponds exactly to the logic of DAST explained in the course.

5.5 Concrete examples of final verification by category of weakness

To maintain the same level of detail as in Part 2, the final validation should be presented vulnerability by vulnerability, or at least category by category.

a) Validation of entries

Initial observation

Some abnormal requests were accepted or insufficiently rejected on the server side.

What needs to be retested

In Postman, send again:

- some required fields are missing;
- invalid formats for email, phone number or username;
- negative or zero amounts;

- inconsistent identifiers in transfer operations.

What needs to be noted

The response must now more firmly reject these entries, ideally with consistent HTTP code and a controlled message, without excessive technical detail.

Expected proof

Postman capture before/after, plus automated Newman test showing that the “invalid payload rejected” scenario now succeeds.

b) Overly verbose error messages

Initial observation

The responses sometimes revealed too much information: requested identifier, overly detailed business reason, or even technical clues.

What needs to be retested

Replay calls on non-existent resources, business errors and edge cases.

What needs to be noted

The message should be more generic, more stable, and not provide unnecessary information to an external observer.

Expected proof

Comparative JSON extract before/after, with commentary explaining the reduction in information exposure.

c) Documentation or exposed surface

Initial observation

Swagger or certain help resources could be too freely accessible.

What needs to be retested



Check access to documentation endpoints according to the environment.

What needs to be noted

Documentation can remain available during development, but its exposure must be justified and controlled. If it is disabled outside of development, this must be explicitly verified.

Expected proof

Browser capture or HTTP request result showing the final state of the exposure.

d) HTTP Headers and Configuration

Initial observation

ZAP reported the absence or weakness of certain security headers.

What needs to be retested

Observe the corrected HTTP responses via ZAP, Postman, or browser.

What needs to be noted

An improvement in responses may be visible: more consistent headers, less permissive behavior, better response discipline.

Expected proof

ZAP capture or capture of response headers.

5.6 Acceptance criteria for a DAST-validated version of DigiBank

At the end of workshop 3, DigiBank can be considered to be properly validated from the DAST point of view if several criteria are met simultaneously.

The first criterion is the absence of major functional regression. The application must still fulfill its main use cases, which aligns with the overall logic of the process: securing it should not mean invalidating its expected functionality.



The second criterion is the effective reduction of prioritized dynamic vulnerabilities. Problematic behaviors observed during the analysis must have disappeared, diminished, or been clearly addressed. It is not required that all ZAP alerts remain, but the pedagogically central problems of the workshop must no longer be visible in their initial form.

The third criterion is the consistency of the remediations with the architecture and with the dynamic observations. A good DAST remediation is not a quick fix. It must correspond to a comprehensible modification of the validation, error handling, endpoint exposure, or HTTP configuration.

The fourth criterion is reproducibility. The proofs must be able to be replayed. A well-constructed Postman collection, a clean Newman pass, and a ZAP scan rerun after correction are essential elements here.

5.7 Capitalization and traceability of results

As in workshop 2, safety only has lasting value if it leaves a clear and understandable record. Therefore, the end of workshop 3 must produce a clear summary of what has been observed, corrected, and revalidated. This summary can take several forms:

- Git commits explicitly stating the security remediations;
- annotated Postman collection;
- clean Postman environment;
- Newman reports exported;
- ZAP captures;
- Summary note or report section describing before/after.

The commits must be meaningful, for example:

- Restrict Swagger exposure to dev profile
- Harden validation on transfer requests
- Standardize API error responses
- Add final DAST validation collection and Newman checks

This traceability requirement is perfectly consistent with the DevSecOps spirit of the course, which links security, automation, reproducibility and continuous improvement.

5.8 Preparing for the next part of the journey

The final validation of Workshop 3 doesn't just conclude an exercise. It explicitly prepares the way forward. The course demonstrates that security doesn't stop at the source code or visible dynamic behaviors; it also extends to dependencies, containers, delivery artifacts, and the overall pipeline. In other words, a DigiBank that is better validated by DAST becomes a better resource for the next workshop.

This preparation takes place on several levels:

- the corrected application provides a cleaner surface for future curing work;
- Postman and Newman scenarios can be reused in a CI/CD pipeline;
- ZAP results can serve as a point of comparison in future iterations;
- The validation discipline acquired here naturally prepares the way to the security of dependencies, Docker images and delivery.

The student must therefore understand that the end of workshop 3 is not an isolated end. It marks the transition from an application that has simply been tested to one that has been observed, corrected, revalidated, and sufficiently mastered to enter a more industrial security approach.

5.9 Expected result at the end of part 5

At the end of this section, the student must be able to demonstrate that the DAST remediations applied to DigiBank have been effectively retested, that they reduce risky behaviors observed at runtime, and that they have not degraded the application's core functionality. They must also be able to produce a coherent evidence package, combining manual testing, Postman/Newman automation, ZAP observation, and a technical justification of the implemented fixes.

In concrete terms, the final version of DigiBank expected at the end of workshop 3 is a version:

- always functional on the main business cases;
- more robust against abnormal inputs;
- less talkative about her mistakes;
- better controlled in its exposed surface;
- dynamically verified in a replayable manner;

- 
- ready to serve as a credible basis for the rest of the DevSecOps journey.

Part 6: Implementing DigiBank's DAST CI/CD Pipeline

6.1 Role of this phase

After manual dynamic analysis, remediation, and final validation, one crucial step remains: industrializing DAST within the CI/CD pipeline. Without this step, dynamic checks remain sporadic, operator-dependent, and difficult to replay properly with each DigiBank update. However, the course emphasizes that security in DevSecOps should not be an exceptional intervention, but a continuous presence, integrated into the development, build, validation, and deployment preparation chain. Therefore, Part 6 aims to transition DigiBank from a dynamically tested application to one that is automatically monitored dynamically, at least within a realistic educational scope.

This phase plays the same role for DAST as the “GitHub Actions” section did for SAST in Workshop 2. It's no longer just about knowing how to use Postman, Newman, or ZAP locally, but about learning how to chain them together in a reproducible workflow. Specifically, the student must understand how to start the application in a pipeline environment, wait for it to become accessible, execute targeted dynamic scenarios, generate actionable reports, retrieve artifacts, and, depending on the chosen policy, terminate the pipeline if certain security conditions are not met.

6.2 DevSecOps Logic of the DAST Pipeline

In the context of this course, the CI/CD pipeline is not simply a technical automation mechanism. It constitutes a control center where compilation, testing, quality assurance, dependency management, packaging, and security are integrated into the same workflow. For DAST, this means that dynamic analysis should not be thought of as a random scan run from time to time, but as a set of checks performed at a coherent point in the cycle: after build, after application startup, before wider deployment, or before promotion to a higher environment.

The DAST pipeline must therefore adhere to several principles. First, it must be reproducible: the same version of DigiBank must be tested in the same way each time it is run. Second, it must be targeted: in an educational context, the goal is not to perform a full penetration test with each push, but to replay a core set of high-value scenarios. Finally, it must be interpretable: the results produced must be understandable to the student, with clearly usable reports, test outputs, and artifacts.

6.3 What the pipeline should automate

DigiBank's DAST CI/CD pipeline must automate an ordered technical sequence. This sequence can be summarized as follows:

1. retrieve the source code;
2. prepare the Java and Maven environment;
3. rebuild DigiBank;
4. start the necessary dependencies, including PostgreSQL if the application requires it;
5. launch the Spring Boot application;
6. wait for the API to actually respond;
7. run the Postman tests via Newman;
8. optionally perform a targeted ZAP scan;
9. publish the reports as artifacts;
10. mark the job as successful or unsuccessful depending on the results.

This pipeline aligns well with the DevSecOps philosophy of the course, which demonstrates that security should be positioned in the pipeline when it is most useful—that is, when the application is fully assembled and executable, but before its wider deployment. It also follows on from Workshop 2, where GitHub Actions was already used to automate builds, tests, and static security checks around DigiBank.

6.4 Part 6 Prerequisites

Before writing the pipeline, several elements must be in place. The first is a version of DigiBank that boots correctly locally, with a clean configuration of Spring profiles and environment variables. This directly extends the best practices already highlighted in Workshop 2 for avoiding hard-coded secrets. The second is the existence of a sufficiently stable Postman DAST validation collection, acquired from previous sections. The third is the availability of a realistic test environment, at a minimum with a database and application backend.

In practical terms, the student should ideally have the following:

- a multi-module Maven DigiBank project that is already functional;
- a PostgreSQL database that can be launched in Docker;

- a Spring profile adapted to the pipeline, for example ci or test;
- an exported Postman collection;
- an exported Postman environment file;
- possibly a ZAP configuration or a simple scan plan;
- a GitHub repository with GitHub Actions enabled.

These prerequisites are important because a DAST pipeline does not analyze "cold" code. It requires a truly executable application, which is reminiscent of one of the structural limitations of DAST presented in the course: unlike SAST, it depends on a launched and observable system.

6.5 Selected Pipeline Strategy

In an educational setting, the most realistic strategy is to build a two-level DAST pipeline.

The first level is a lightweight, systematic, and rapid dynamic check, executed with each significant push or pull request. It relies primarily on Newman and replays a Postman collection of security regressions. This level verifies that the main dynamic protections remain in place: server validations, controlled error handling, rejection of certain abnormal cases, and stable behavior of critical endpoints.

The second level is a more intensive, less frequent check, performed, for example, on the main branch, on an integration branch, during a nightly build, or before a demonstration. This level can include OWASP ZAP with a targeted scan or a baseline scan on the URL exposed by DigiBank. This distinction is consistent with the course, which emphasizes that all checks should not be concentrated in one place, nor should security become a constant obstacle; some checks should be frequent and quick, others more thorough but less systematic.

6.6 Organization of expected files in the repository

```
digibank-parent/  
├── .github/
```

```
| └─ workflows/

| └─ digibank-quality.yml

| └─ digibank-dast.yml

└─ dast/

| └─ postman/

|   └─ DigiBank-DAST-Validation.postman_collection.json

|   └─ DigiBank-local.postman_environment.json

| └─ zap/

|   └─ zap-baseline-rules.conf

|   └─ zap-ignore.conf

| └─ reports/

└─ docker-compose.yml

└─ pom.xml

└─ ...
```

This structure is important from a pedagogical standpoint. It demonstrates that DAST-related elements should not be scattered randomly throughout the repository, but treated as genuine project artifacts: test collections, environment, scan rules, and future reports. This extends the traceability logic already highlighted in Workshop 2 with Git, Maven, and GitHub Actions.

6.7 Preparing the Postman collection for the pipeline

Before integrating Newman into GitHub Actions, the Postman collection must be made usable in non-interactive execution. This involves several points.

First, all requests must use environment variables rather than hard-coded URLs. For example:

- `{{baseUrl}}/api/customers`
- `{{baseUrl}}/api/accounts`
- `{{baseUrl}}/api/transfers`

Next, the scenarios must be as robust and self-contained as possible. Dependence on manually created identifiers within the tool should be avoided. When possible, a query should create what the next one needs, then store the necessary values in collection or environment variables. Examples of scripts:

```
const json = pm.response.json ( ) ;

if ( json && json.id ) {

pm.collectionVariables. set ( "customerId" , json.id);

}
```

Or again:

```
pm.test ( " The HTTP status is 201" , function () {

pm.response.to.have. status ( 201 );

} );

const body = pm.response.json ( ) ;

p.m. expect ( body ). to . have . property ( "id" );
```

```
p.m. collectionVariables . set ( "accountId" , body . id );
```

Explicit security tests should also be included in Postman scripts. Examples:

```
pm.test ( " The invalid request is rejected" , function () {  
  pm.expect (pm.response.code).to.be. oneOf ( [ 400,404,422 ] )  
  ;  
} );  
  
pm.test ( " The response contains no technical trace" , function  
( ) {  
  const text = pm.response. text (). toLowerCase ();  
  pm.expect ( text ).to.not. include ( "exception" );  
  pm.expect ( text ).to.not. include ( "stacktrace" );  
  pm.expect ( text ).to.not. include ( "org.springframework" );  
} );
```

This preparation is essential because, in the pipeline, Newman becomes the first automated level of the DAST. Therefore, it should not be provided with a purely exploratory collection or one dependent on manual manipulation.

6.8 Spring environment preparation for the pipeline

One of the most critical aspects of a DAST pipeline is ensuring a reliable application startup. For DigiBank, this requires a suitable Spring configuration. As demonstrated in

Workshop 2, secrets and sensitive parameters should be passed via environment variables rather than hard-coded values in version control files.

An `application-ci.yml` file or equivalent configuration can be provided in `digibank-web/src/main/resources/`. Example approach:

```
spring :  
  
  datasource :  
  
    url : ${SPRING_DATASOURCE_URL}  
  
    username : ${SPRING_DATASOURCE_USERNAME}  
  
    password : ${SPRING_DATASOURCE_PASSWORD}  
  
    driver-class-name : org.postgresql.Driver  
  
  jpa :  
  
    hibernate :  
  
      ddl-auto : validate  
  
      show-sql : false  
  
  flyway :  
  
    enabled : true  
  
server :  
  
  port : 8080  
  
error :  
  
  include-message : never  
  
  include-stacktrace : never
```

```
springdoc :  
  
  api-docs :  
  
    enabled : false  
  
  swagger-ui :  
  
    enabled : false
```

This configuration is fully in line with the recommendations of TP2 on the separation between versioned configuration and sensitive values, as well as on the reduction of unnecessary exposure such as Swagger out of appropriate context.

6.9 First level: GitHub Actions pipeline with Newman

The first level of the DAST pipeline should be simple, robust, and frequent. The central tool here is Newman. A GitHub Actions workflow can be written as follows:

```
Name : DigiBank DAST Pipeline  
  
on :  
  
  push :  
  
    branches : [ main , master , develop ]  
  
  pull_request :  
  
    branches : [ main , master , develop ]  
  
jobs :
```

```
dast-newman :

  runs-on : ubuntu-latest

  services :

    postgres :

      image : postgres:16

      approx .

      POSTGRES_DB : digibankdb

      POSTGRES_USER : digibank

      POSTGRES_PASSWORD : digibank123

      ports :
- 5432:5432

      options : > -

--health-cmd="pg_isready -U digibank -d digibankdb"

--health-interval=10s

--health-timeout=5s

--health-retries=5

  steps :
- name : Checkout repository

  uses : actions/checkout@v4
```

```

- name : JDK 17 Setup

  uses : actions/setup-java@v4

  with :

    distribution : temurin

    Java version : 17

    cache : maven

- name : Build DigiBank

  run : mvn clean verify -DskipTests

- name : Start DigiBank

  run : |

nohup mvn spring-boot:run -pl digibank-web > digibank.log 2>&1 &

  approx .

  SPRING_PROFILES_ACTIVE : here

                                SPRING_DATASOURCE_URL :
jdbc:postgresql://localhost:5432/digibankdb

  SPRING_DATASOURCE_USERNAME : digibank

  SPRING_DATASOURCE_PASSWORD : digibank123

- name : Wait for application

```

```
run : |

for i in {1..30}; do

if curl -s http://localhost:8080/actuator/health || curl -s
http://localhost:8080; then

echo "Application is up"

exit 0

fi

sleep 5

done

echo "Application did not start in time"

exit 1


- name : Set up Node.js

  uses : actions/setup-node@v4

  with :

    node-version : 20


- name : Install Newman

  run : npm install -g newman newman-reporter-html


- name : Run Newman DAST suite

  run : |
```

```

mkdir -p dast/reports

newman run
dast/postman/DigiBank-DAST-Validation.postman_collection.json \
-e dast/postman/DigiBank-local.postman_environment.json \
--env-var baseUrl=http://localhost:8080 \
--reporters cli,html,json \
--reporter-html-export dast/reports/newman-report.html \
--reporter-json-export dast/reports/newman-report.json

- name : Upload Newman reports

  uses : actions/upload-artifact@v4

  with :

    name : newman-dast-reports

    path : dast/reports/

- name : Upload application log

  uses : actions/upload-artifact@v4

  if : always()

  with :

    name : digibank-app-log

    path : digibank.log

```



This pipeline is very important from a pedagogical standpoint because it concretely demonstrates how DAST can be integrated into the CI/CD pipeline. It rebuilds the application, launches PostgreSQL, starts DigiBank, waits for it to be available, runs dynamic tests, and publishes the reports. This logic perfectly embodies the course's vision that security must be present in the CI/CD chain from the moment the application is actually assembled and testable.

6.10 Observations to be made on this pipeline

As in part 2 of workshop 2, it is necessary to explain what to observe in the pipeline, and not just provide YAML.

a) Observation on application startup

The student must check:

- that PostgreSQL is correctly declared as a GitHub Actions service;
- that the application uses the correct environment variables;
- that `mvn spring-boot:run -pl digibank-web` starts without error;
- that the waiting phase does not question the application too early.

If the application fails to start, the DAST pipeline is unusable. The `digibank.log` file then becomes essential evidence to review.

b) Observation on Newman

We need to check:

- that the collection correctly uses `{{baseUrl}}`;
- that HTML and JSON reports are generated;
- that Postman tests truly fail if an expected behavior disappears;
- that security cases are not mixed haphazardly with purely functional cases.

The expected observation here is important: Newman doesn't "do security" on its own. It executes what has been explicitly formalized in the collection. If the tests are weak, the pipeline will be weak.

c) Observation of the artifacts

The pipeline needs to upload:

- the Newman report HTML;
- the Newman JSON report;
- the application startup log.

This publication of artifacts is consistent with the logic of proof and traceability emphasized throughout the workshops, for both SAST and DAST.

6.11 Second level: OWASP ZAP integration into the pipeline

Once the first level is stable, a second, more exploratory level can be added using ZAP. Here, it's important to be realistic: in an educational setting, a baseline scan or a targeted scan is more appropriate than an aggressive, exhaustive scan.

An example of a GitHub Actions job can be provided:

```
dast-zap :  
  
  runs-on : ubuntu-latest  
  
  needs : dast-newman  
  
  steps :  
- name : Checkout repository  
  uses : actions/checkout@v4
```

```
- name : Run OWASP ZAP Baseline Scan

  uses : zapproxy/action-baseline@v0.12.0

  with :

    target : 'http://host.docker.internal:8080'

    cmd_options : '-a -j -m 5 -r zap-report.html'
```

In a real-world GitHub Actions pipeline, access between containers and the application must be carefully planned. Another, often more stable, strategy is to run DigiBank itself in Docker Compose and then launch ZAP against the exposed service on a shared network. The key takeaway is this: ZAP in the pipeline should not be added as a black box, but as a tool positioned to verify the observable surface of the application after startup.

6.12 Cleaner variant with Docker Compose

For a more robust DAST pipeline, especially if DigiBank relies heavily on PostgreSQL and a precise startup order, a Docker Compose approach may be preferable. The repository can contain a file such as:

```
version : '3.9'

services :

  postgres :

    image : postgres:16

    environment :
```

```

    POSTGRES_DB : digibankdb

    POSTGRES_USER : digibank

    POSTGRES_PASSWORD : digibank123

    ports :
- "5432:5432"

digibank :

    build : .

    environment :

        SPRING_PROFILES_ACTIVE : here

                                SPRING_DATASOURCE_URL :
jdbc:postgresql://postgres:5432/digibankdb

        SPRING_DATASOURCE_USERNAME : digibank

        SPRING_DATASOURCE_PASSWORD : digibank123

    ports :
- "8080:8080"

    depends_on :
- postgres

```

In this case, the pipeline can:

- run docker compose up -d --build ,
- wait for <http://localhost:8080>,
- execute Newman

- run ZAP,
- publish the reports.

This variant is very consistent with the rest of the course, which extends the security posture to containers, images, and delivery artifacts. It also naturally prepares the way for the workshop dedicated to the security of containers and dependencies.

6.13 Pipeline Failure Conditions

A DAST pipeline should not fail randomly. What constitutes a failure must be clearly defined.

For the Newman level, the easiest way is to make the job fail if:

- a Postman test fails;
- The application does not start;
- An expected report has not been produced.

For the ZAP level, the approach must be more cautious. In a teaching workshop, it is often preferable to:

- either to publish the report without immediately blocking the pipeline;
- either to block only on certain signals considered critical;
- either to start with an observation mode before introducing a blocking mode.

This gradual approach is consistent with the DevSecOps spirit of the course: integrating security into the pipeline does not mean making it abruptly unusable, but building actionable and proportionate safeguards.

6.14 What to analyze in the CI/CD DAST results

As in the other parts, the student needs guidance on what to read in the pipeline results.

a) Newman Results

It is important to observe:

- the total number of queries executed;
- the number of tests passed/failed;
- explicitly validated security scenarios;
- potential regressions after a code change.

b) ZAP Results

It is important to observe:

- recurring alerts;
- new alerts that appeared after a modification;
- the difference between useful signal and tool noise;
- remarks related to headers, surface exposure and responses.

c) Log results

DigiBank logs should be reviewed if:

- the pipeline never reaches the Newman stage;
- some roads are not responding;
- Flyway, JPA, or the datasource prevents startup.

Here again, the pipeline is worthless if it is not interpreted. The course specifically emphasizes the idea that security tools only make sense if the results are understood, prioritized, and used in a technical decision.

6.15 Gradual integration with the rest of the quality pipeline

Following on from workshop 2, part 6 should also demonstrate that the DAST pipeline does not replace the SAST pipeline. It complements it. A coherent chain around DigiBank could therefore look like this:

- build and unit tests;
- SAST and dependency analysis;
- application packaging or launch;
- DAST light with Newman;

- DAST is broader with ZAP depending on the context;
- publication of reports and artifacts.

This structure directly reflects the complementarity between SAST and DAST highlighted in the course: the former acts on the software's structure, the latter on its execution behavior. In a modern pipeline, the two approaches should interact rather than be mutually exclusive.

6.16 Expected result at the end of part 6

By the end of this section, students should be able to implement a functional DAST CI/CD pipeline for DigiBank, based at a minimum on GitHub Actions, automated application startup, execution of a Postman collection via Newman, and publication of reports as artifacts. They should also understand how to progressively add a second level of dynamic control with ZAP, how to interpret the results, and how to position these controls within a DevSecOps framework consistent with the rest of the course.

Specifically, the expected release of this part is a version of DigiBank for which:

- The essential dynamic tests are automatically replayed;
- DAST evidence is published via the pipeline;
- Simple security regressions can be detected without systematic manual testing;
- The application is better prepared for continuous security, integrated into the delivery flow.



Part 7: Final validation, capitalization of DAST results and preparation for the next stage of the process

After identifying the dynamic vulnerabilities observable in DigiBank, methodically executing security tests on the live application, and remediating the identified weaknesses, Workshop 3 must conclude with a final validation phase, consolidating evidence and structuredly preparing for the next steps in the DevSecOps process. This last part is essential because a DAST workshop cannot be considered complete simply because alerts were raised by OWASP ZAP, a few requests were modified in Postman, or some application responses were deemed abnormal. It is still necessary to demonstrate that the observed vulnerabilities were correctly interpreted, that the applied fixes effectively reduce the risk without breaking the expected functional behavior, and that DigiBank emerges from this workshop as a more robust, better understood, and better-prepared application for subsequent security audits. This logic is fully consistent with the course, which presents DAST as an analysis of the application's actual behavior at runtime, complementary to SAST and useful for confirming, refining, or revealing weaknesses that only become visible through interaction with requests, sessions, errors, and the system's actual exposure surface. It is also consistent with the DigiBank case study, which emphasizes the need to observe not only the code, but also the APIs, error messages, session mechanisms, access controls, and configurations actually exposed at runtime.

7.1 Role of final validation in workshop 3

The final validation aims to confirm that the work carried out in Workshop 3 was not limited to simple tool exploration or an accumulation of observations, but that it led to a real improvement in DigiBank's dynamic security posture. From a pedagogical perspective, this step is crucial, as it shifts the student from a logic of occasional observation to one of mastery. Running a DAST scan or triggering an HTTP error does not yet constitute securing an application. Securing, within the context of this workshop, means understanding what application behavior reveals, distinguishing truly problematic



anomalies from minor side effects, correcting what needs to be corrected, and then verifying that the corrected application remains consistent, stable, and defensible.

This final phase is also crucial because DAST fixes often affect sensitive areas of the system. These can include input validation, HTTP responses, error serialization, token management, protected routes, CORS configuration, cookies, or security headers. A poorly designed fix can make an alert disappear while introducing a functional regression, an API inconsistency, or a new vulnerability. Therefore, in the spirit of DevSecOps, the end of the workshop should not only confirm that a problem has been identified, but that it has been properly addressed, rechecked, and integrated into a continuous improvement process.

7.2 What needs to be rechecked after remediation

Following the remediation phase, several dimensions must be explicitly rechecked. The first concerns DigiBank's functional and technical stability. The application must always start correctly, expose its expected endpoints, handle essential business scenarios, and respond in a manner consistent with legitimate use cases. An application that becomes quieter in terms of error messages but no longer allows for customer creation, account opening, or successful transfer execution would not meet the workshop's objectives. This requirement directly extends the logic of the process, in which security should never be considered as a barrier to the software, but rather as an integral part of its overall quality.

The second dimension concerns the dynamic vulnerabilities themselves. Weaknesses deemed priorities in the previous sections must have disappeared, been mitigated, or clearly defined after remediation. This does not mean requiring a clean OWASP ZAP scan or the absolute absence of any anomalies, as some signals depend on the context, limitations of the test environment, or the depth of the scan. However, truly relevant vulnerabilities, such as an overly revealing error response, insufficient server validation, an accessible critical route without adequate protection, or a missing security header on a sensitive exposure, must no longer exist in their original form. The course specifically reminds us that the DAST is used to observe weaknesses visible at runtime and should be interpreted with discernment, not mechanically reproduced.

The third dimension concerns the quality of security reasoning. In a DevSecOps workshop, remediation is not judged solely on the raw output of the tool. Students must be able to explain why a particular vulnerability was deemed critical, why a specific fix was applied, why a particular behavior was reclassified as a secondary risk, and how the



corrected version actually improves DigiBank's security posture. This ability to reason is central to the case study, which emphasizes the need to prioritize risks and build a coherent security response rather than a collection of isolated actions.

7.3 Expected technical checks

Final validation must rely on a clear sequence of technical checks. DigiBank must first be restarted in a clean, stable, and reproducible environment. The endpoints must be retested with standard functional scenarios to ensure that the remediation measures have not broken the expected behavior. This verification can be performed using Postman and, more importantly, Newman, which allows for the systematic replay of the API collection validating the main business cases. This approach is consistent with the logic of reproducibility and automation already present in previous workshops and in the project's industrialization.

Next, the dynamic security tests must be rerun. The same malformed inputs, parameter manipulations, sensitive route tests, HTTP response observations, and ZAP scans must be executed again to compare the state before and after the fix. The goal is not just to confirm a visual improvement, but to have verifiable evidence. If, before the fix, the application returned error messages detailing internal identifiers, accepted unexpected server-side values, exposed insufficient headers, or responded excessively on certain routes, the fixed version should demonstrate a concrete improvement in these same areas.

Finally, this validation must also consider the consistency between DAST and SAST. The course clearly reiterates that SAST and DAST are complementary, one observing the internal structure of the software, the other its external behavior during execution. A vulnerability revealed dynamically must therefore lead to verification that its cause has been corrected in the code, the configuration, or the business logic. In other words, the final phase of workshop 3 is not limited to simply noting that a symptom has disappeared. It must also ensure that the fix implemented in DigiBank is technically sound, maintainable, and compatible with the rest of the course, particularly the workshop dedicated to dependencies, containers, and artifacts.

7.4 Expected technical checks

A final revalidation sequence can be presented in a simple and structured way. The exact commands depend on the local project organization, but the expected logic remains the same: rebuild, restart, replay the automated functional tests, and then replay the DAST observations.

Example of a final validation sequence:

```
CD digibank-parent

mvn clean test

mvn clean verify

docker compound up -d

newman          run          DigiBank.postman_collection.json          -e
dev.postman_environment.json
```

After this phase, OWASP ZAP can be restarted either manually via its interface or automatically if an execution script has been prepared in the lab. The key here is not to impose a single universal command, but to emphasize that the final validation must always follow a reproducible logic: restart the application, verify legitimate business processes, and then replay the attack or disruption scenarios to compare the results before and after remediation. This idea directly aligns with the DAST's positioning in the course, where dynamic analysis is described as a validation of the application's actual exposure surface while it is running.

7.5 Acceptance criteria for a corrected version of DigiBank

A corrected version of DigiBank can be considered acceptable at the end of Workshop 3 if several criteria are met simultaneously. The first criterion is the absence of major functional regressions. Essential business operations must still function correctly. Security must not hinder the service provided by the application, but rather enhance it.



This requirement remains constant throughout the DevSecOps curriculum presented in the course.

The second criterion is the effective reduction of prioritized dynamic vulnerabilities. The anomalies deemed most significant at the time of analysis must have disappeared, been significantly reduced, or be accompanied by a clear technical justification when their correction is not yet fully within the scope of the workshop. A simple cosmetic reformulation without real risk reduction is insufficient. For example, hiding an error message on the front end while maintaining dangerous server-side logic would not constitute satisfactory remediation.

The third criterion is the architectural and operational consistency of the fixes. Remediation must integrate seamlessly with controllers, services, filters, exception handlers, HTTP configurations, and security components. It must not transform DigiBank into an opaque, fragile, or difficult-to-maintain system. A good DAST fix is one that reduces the risk observable at runtime while remaining readable, stable, and compatible with future developments. This consistency is particularly important in an application like DigiBank, which serves as a guiding principle throughout the course, covering code, execution, dependencies, and deployment.

The fourth criterion is the student's ability to demonstrate the relationship between observation, correction, and result. The workshop aims not only for a technically modified version of DigiBank, but also for a version whose transformations are understood, justified, and traceable. This requirement aligns with the overall spirit of the course, which emphasizes the interpretation of security results and their reasoned integration into a DevSecOps approach.

7.6 Capitalization and traceability of results

The end of workshop 3 must also include a knowledge-sharing component. In a DevSecOps approach, it's not enough to simply fix things; a clear record must be kept of what was observed, what was corrected, what was revalidated, and any remaining limitations. This knowledge-sharing transforms the one-off exercise into lasting learning. It also facilitates continuity between workshops, since the dynamic observations made here contribute to a broader understanding of DigiBank's security posture.

This knowledge sharing can take several complementary forms. First, Git commits must clearly reflect the remediation intentions, for example, regarding server validation, error handling, security headers, or access to sensitive endpoints. Second, the workshop



report must explicitly state, for each primary vulnerability, the initial observation, the evidence of observation, the applied fix, and the result after revalidation. Finally, reports or captures from Newman, OWASP ZAP, Postman, and HTTP responses must be organized in a readable manner to create a coherent evidence package.

Within an integrated security framework, this traceability is not merely of academic value. It also prepares the ground for professional practices such as review, internal auditing, release validation, and communication between development, operations, and security. The course specifically emphasizes this vision of security as a distributed, progressive practice integrated into the software development lifecycle.

7.7 What the student should remember from this final phase

By the end of this section, students should understand that a DAST workshop is not simply about running OWASP ZAP, triggering a few application errors, or passively observing abnormal responses. They should remember that dynamic analysis aims to observe the application as it actually behaves in operation, to confirm or reveal vulnerabilities visible from the outside, and then to guide a reasoned and verifiable remediation. This understanding directly extends the objectives of the DAST session in the course, which emphasizes the value of observation in a runtime context and the limitations of an analysis that is not interpreted intelligently.

It should also be remembered that application security is strengthened when multiple perspectives complement each other. SAST helps identify certain weaknesses in the code structure. DAST reveals what the application discloses or allows to happen while running. The DigiBank case study specifically highlights this cross-cutting nature of risk, which permeates the code, execution, dependencies, containers, and architectural choices. The conclusion of workshop 3 should therefore emphasize a simple yet crucial idea: effective security is not just about detection; it's about understanding, correcting, revalidating, and methodically preparing for the future.

7.8 Expected result at the end of part 7

At the end of Part 7, the student should not only have a dynamically tested DigiBank application, but also a revalidated version, better understood and documented from a security perspective, observable at runtime. Specifically, they should be able to explain why



certain HTTP responses were deemed too verbose, why certain server validations needed strengthening, why certain routes or behaviors posed a risk, how remediation measures were implemented, and how the final revalidation confirmed the system's effective improvement.

DigiBank thus becomes a more credible foundation for the rest of the course. After being observed statically in workshop 2, then dynamically in workshop 3, the application is now better prepared for the analyses related to dependencies, containers, artifacts, and deployment mechanisms covered in the following steps. This progression perfectly reflects the overall logic of the course: prevent problems early, verify in context, correct them judiciously, clearly document them, and then progressively strengthen the entire software chain.